

Hop3 — Application Experience Reports

Twenty applications packaged across three build paths, and what each one cost the platform

2026-08-01 · DRAFT

Contents

Aggregate Experience Report: Packaging 20 Applications for Hop3	2
Experience Report: BookStack	8
Experience Report: Bugsink	10
Experience Report: Dolibarr	13
Experience Report: Easy!Appointments	15
Experience Report: Forgejo	17
Experience Report: Gitea	19
Experience Report: Invoice Ninja	21
Experience Report: Isso	23
Experience Report: Kanboard	25
Experience Report: Keycloak	27
Experience Report: LimeSurvey	29
Experience Report: Matomo	31
Experience Report: Mattermost	33
Experience Report: Miniflux	35
Experience Report: Nextcloud	37
Experience Report: Paheko	39
Experience Report: Radicale	41
Experience Report: Uptime Kuma	43
Experience Report: Vikunja	45
Experience Report: WordPress	47

Aggregate Experience Report: Packaging 20 Applications for Hop3

Status: Final (0.7): the per-app reports are `report_status: final`; corrections continue against a published baseline.

Last updated: 2026-07-31

Overview

Twenty open-source applications are packaged for Hop3 and published in the signed catalog. Each is packaged for up to three build paths (native, hand-written Nix, and Nix generated from a template) and each carries a `check.py` that signs in through the application's own authentication and confirms a wrong password is refused.

Packaging is system-validation work. Every application was chosen to stress a different edge of the platform, and the per-app reports record which edge; where an app merely confirmed existing behaviour, its report says so without inventing a contribution.

The instrument matters more than the count. "Passing" here means an authenticated sign-in through the application's own surface, with a wrong password refused. That bar was adopted only after a majority of the corpus was found to clear the previous one (HTTP 200) while being impossible to log into. Read against that bar, five of these twenty were at one point serving with no usable credential at all, and every instrument the project then had reported them working.

Application coverage

The table is derived from the reports' own headers, which `hop3-tools catalog reports` checks against the recipes, so it cannot drift from them silently. `n/a` means there is no recipe for that variant. Docker is not listed: recipes exist for most of these applications and none has been measured at this bar, so the report does not claim the variant.

App	Language	Database	Native	Nix	Nix-gen
BookStack	php	mysql	pass	pass	pass
Bugsink	python	postgres	pass	pass	pass
Dolibarr	php	postgres	pass	pass	pass
Easy!Appointments	php	mysql	pass	n/a	fail
Forgejo	go	postgres	pass	pass	pass
Gitea	go	postgres	pass	pass	pass
Invoice Ninja	php	mysql	pass	pass	pass
Isso	python	none	pass	pass	pass
Kanboard	php	mysql	pass	pass	pass
Keycloak	java	postgres	pass	pass	pass
LimeSurvey	php	postgres	pass	pass	pass
Matomo	php	mysql	pass	pass	pass

App	Language	Database	Native	Nix	Nix-gen
Mattermost	go	postgres	pass	pass	pass
Miniflux	go	postgres	pass	pass	pass
Nextcloud	php	mysql	pass	n/a	pass
Paheko	php	none	pass	pass	pass
Radicale	python	none	pass	pass	pass
Uptime Kuma	node	none	pass	n/a	n/a
Vikunja	go	postgres	pass	pass	pass
WordPress	php	mysql	pass	n/a	pass

Languages: php (10), go (5), python (3), java (1), node (1)

Databases: postgres (9), mysql (7), none (4)

Five applications were packaged, reported, and later dropped from the corpus: Adminer, Focalboard, Grafana, Jenkins and Wiki.js. Their reports are kept under `withdrawn/`, marked `report_status: withdrawn`, because the packaging work was real even though the applications are no longer advertised.

What the numbers mean, and what they do not

Native is the variant the catalog publishes, and it is 20 of 20 on a complete recorded run (2026-08-01). Earlier drafts of this report declined to claim it: the runs before this one topped out at 19 of 20, every application had been seen green at some point, and no single run had been all-green *and* recorded. A claim resting on “each of them passed at some point” is a weaker object than one resting on a run that can be pointed at, so the number waited for the run. This is that run.

Nix-gen is 18 of 19 on that run. Easy!Appointments is its single failure: it builds its login form in JavaScript, so neither the HTTP check nor the browser harness can complete a sign-in. Its report records the failure. That failure is stable and understood.

Hand-written Nix is 16 of 16 on a complete recorded run of its own (2026-08-01, browser harness). It read 15 of 16 earlier the same day, and the intervening hour is the more useful part of the record.

Bugsink had not installed. The reading that had gone unchallenged for weeks was that it was slow: three successive `start-timeout` increases across the three variants (120 → 180 → 240) had been spent on a failure whose headline was “did not start within Ns”. It was not slow. Its gunicorn master bound the port eight seconds in and then no worker ever booted: no “Booting worker” line, no traceback, a silent hang for the remaining 232 seconds. The platform had diagnosed it correctly at every occurrence, in the line under the headline: *“the app’s port is listening but it did not answer an HTTP request: the server bound its socket but no worker is serving.”*

The cause was structural. **The hand-written recipe ran Bugsink as one process, and Bugsink is two.** Its two siblings each migrate a second database (`migrate --database=snappea`) and declare a second worker (`[run.workers] snappea = "bugsink-runsnappea"`), both carrying comments saying the application does not work without them; the hand-written recipe did neither, while generating its config from the same `--template docker` that assumes both. It could not have done otherwise: its setup lived inside the `web` start wrapper, and anything in that file runs only for gunicorn, so setup was unshareable with a second process by construction. Moving it to `before-run` and declaring the worker took the application from a 245-second hang to a 40-second pass.

Native holds the corpus's only *20-application* all-green run; both figures now rest on complete recorded runs.

Both families were measured for the first time that week, and the first measurement of the hand-written corpus **failed all sixteen applications**, three days after a deploy-oriented matrix scored the recipes in its scope 15 ok, 0 fail. Two instruments, one corpus, opposite verdicts. That is the result the sign-in bar exists to produce. The older instrument could not see what it was reporting green; the same corpus reached 16 of 16 within a day.

What the first measurement found

Five of the sixteen were serving with no usable credential, in five different ways. Radicale's [auth] type defaulted to none and nothing set it, so every calendar and address book was public. Isso's config had no [admin] section, so its moderation dashboard was served disabled. Miniflux and Keycloak each shipped a literal changeme (an identity provider among them), so the deployed instance had an administrator whose password is in this repository while the operator held one that did not work. LimeSurvey installed itself as `console.php install admin password123` with the result discarded by `2>/dev/null || true`: a published password *and* a failed install reported as success, in one line.

No status assertion and no content assertion can distinguish any of those from a working deployment. The argument is not hypothetical.

The remainder divide cleanly: two forges carried a *native* create command naming `./gitea`, a path with no meaning in a Nix layout; three carried secrets minted by a wrapper that re-evaluates them on every start; three were served straight out of the read-only store and so could never be installed at all; and the rest had no first-run bootstrap, because these recipes predate ADR 056 and the variant generator grafts an identity they cannot honour.

One methodological finding

Every application in this corpus exists in two to four packagings, and the fastest route to a defect in one of them is a diff against a packaging that already passes. The last four hand-written failures each took a single attempt once that was the method, having resisted several rounds of reasoning from symptoms.

The wrong theories are kept because each would have sent someone into the platform after a defect that is not there: a reverse-proxy misconfiguration (Matomo actually needed the `core:update` both working variants run), a version skew (`nix eval` says `nixpkgs` ships the *same* Vikunja the template variant compiles), and PHP's built-in server mismatching responses under concurrency (every working variant uses `php -S` too). Reading the redirect target, the log line, and the working recipe would each have been quicker than any of them.

Docker is out of scope for this report. Recipes exist for most of these applications and no run has ever measured one at the sign-in bar, so the corpus has nothing to say about them. Earlier versions of these reports recorded Docker as "Passing", which is the specific rot this format exists to prevent: that status was asserted without observation. The variant is not claimed; on the evidence above, whoever measures it first should expect to find things.

The bar, and why it moved

For most of this corpus's life, "passing" meant the application deployed and answered HTTP 200. That bar was retired after a majority of the catalog was found to clear it while being impossible to log into: apps served their login page perfectly and rejected every credential. Some of those failures were the platform's,

some the recipes’, and one was the checking library posting an empty form body: the same symptom for all three.

Reports now name the bar they were verified at, in a header field the checker reads:

Bar	Meaning
http-status	returned 200; not sufficient for a catalog app
http-content	served its own content (a <code>contains</code> assertion)
authenticated	signed in through the app’s own auth; a wrong password was refused

An advertised application may not be reported final on anything less than authenticated, and the checker enforces it.

One gap remains open at the platform level: `[healthcheck].contains` is declared by no recipe in the corpus, and a deploy currently treats any status line as “serving”. So the “App is now running” a deploy prints is satisfied by a 500. Until that changes, a deploy’s own report of success is a weaker claim than these reports are.

Recurring technical findings

The per-app reports carry the detail; these are the patterns that showed up in more than one.

An app’s binary is rarely named after the app, and never the same in two packagings. `buildGoModule` names it after the module path element, so a source build gives `forgejo.org`, `miniflux.app`, `api.nixpkgs` names the same three `gitea`, `miniflux`, `vikunja`; its Mattermost derivation ships no `mmctl` at all, though every `mmctl --local` in that app’s documentation assumes one. `$out/bin` holds only the generated wrapper, which execs one fixed subcommand, so putting it on `PATH` does not help; `${pkg}` is the stable binding for the application’s own derivation. A command copied from a sibling recipe fails differently, without working, and `nix eval` on the pinned `nixpkgs` settles the question in seconds.

A credential is not injected until something maps it. Hop3 generates an administrator password, stores it encrypted, and injects `HOP3_ADMIN_*`. An application reads whatever *it* reads, and the mapping is the recipe’s job (`[env.computed]`). Where the mapping was missing, the recipe usually still carried a literal default, so the deployed application had an administrator whose password was published in the repository, while the operator was handed one that did not work. The smoke test reported only the second half of that. Four recipes were in this state.

A secret evaluated by the wrapper is a secret that rotates. Values in `[nix.env-exports]` are shell expressions re-evaluated on every start. Several recipes minted signing keys there with `$(head -c 32 /dev/urandom | base64)`, which silently invalidates every session on restart and, for the forges, makes 2FA secrets and stored credentials undecryptable. Generated-once `[env]` secrets are the mechanism; the shell is not.

The generated wrapper is not the only thing that runs the app’s code. `[run]` `before-run` and the `[admin]/[probe]` create commands execute directly. Anything that lives only in the wrapper (`LD_LIBRARY_PATH` from `nix-runtime-libs`, a composed `DATABASE_URL`) is absent there, and the failure looks like a broken application, not a missing environment.

PHP `__DIR__` resolves symlinks, so a Nix store path reached through one lands back in the read-only store. `needs-writable-dir` copies the tree instead, and the copy’s *timing* was the single largest cause of `nix-gen` failure. It happened when the app started, which is after `before-run`, so any application whose

bootstrap needs its own code found an empty directory. Materialising it at deploy time, ahead of the first before-run command, moved six applications at once.

An application’s public address is not the address it binds. Four recipes pinned theirs to `http://localhost:8080` (three in `[nix.runtime-env]`, baked into the wrapper at build time) and one built it by hand as `http://${HOST_NAME}`, right host, wrong scheme. The consequences differ (a Vue frontend calling the visitor’s own machine; a Laravel redirect loop; a Keycloak console whose sign-in redirect points nowhere; a JavaScript form posting over http from an https page) and the shape is identical: the server side is unaffected, so every status and content assertion passes while a browser is shown nothing usable. `HOP3_PUBLIC_URL` is injected once the hostname is settled and reaches config files, before-run and create; make `validate` now rejects a hand-built public URL in any build-time table.

A git tag is not a release. Paheko vendors the KD2 framework into its release tarball without committing it; Easy!Appointments’ tag omits both its minified assets and its vendor/ tree. `fetchFromGitHub` on the tag produces a package missing part of the application, and the failure surfaces at runtime as a missing `require_once`, not at build time.

A setting that lives in a shell script does not survive repackaging. `DISABLE_REGISTRATION = true` sat in the native recipes’ `scripts/setup-config.sh`, and no Nix variant carries a `scripts/` directory, so all four Nix forge builds put an internet-facing Gitea or Forgejo online on which the first visitor could register. It shipped for as long as those variants existed. **The sign-in bar does not catch it:** an application with open registration signs in perfectly and refuses a wrong password. The bar is a floor, and a security posture that lives only in a file one packaging happens to carry is not a posture.

A bootstrap CLI’s exit code is not its outcome. Gitea and Forgejo print `admin user create`’s refusal and exit 0; Dolibarr’s install steps do the same; Vikunja’s `user` command answers an unknown subcommand with usage text and exit 0, so calling one that does not exist is a silent no-op. Every `create` in this corpus now verifies the account exists (by listing it, or by querying the row), because the platform’s own “admin created” message is a promise it cannot otherwise keep.

A read-only store is not a place an application can be installed. Three recipes served straight out of `$out`, which means no schema, no config, no uploads and no account: the app deploys, starts, answers HTTP, and cannot be signed into. The failure presents as an application-level message (“please run setup”, “the username is required”), which points at the app, not at the filesystem underneath it.

Multi-process applications are under-served. Bugsink needs a snap pea worker and a second `migrate --database=` before any post-login page works; Nextcloud wants a cron worker; Invoice Ninja wants a queue. `[run.workers]` expresses the processes, but not per-process environment, ports, or limits.

Reproducibility tiers

Every tier builds in a sealed sandbox: the dependency set is vendored into a fixed-output derivation from a committed lockfile, so the package manager runs offline. The tiers rank provenance; hermeticity is not the measure.

Tier	Method	Rebuilds identically?	Auditable to source?	Multi-arch?
1	nixpkgs package	yes	yes	yes
2	source build against a committed lockfile	yes	yes	one arch per lockfile

Tier	Method			Rebuilds identically?	Auditable to source?	Multi-arch?
3	pre-built (fetchurl)	upstream	artefact	yes	no	no

The pre-built-binary problem earlier drafts described has largely been worked through: Gitea, Forgejo, Miniflux and Vikunja are compiled from source with go-source, and Mattermost and Keycloak are nixpkgs-wrapper. Run `hop3-tools nix tiers apps/real-apps-nix-gen` for the current split, and see ADR 008 for the assessment.

Two moved the other way, deliberately. Paheko and Easy!Appointments now fetch the upstream *release* archive. Building from the git tag was dropped because the tag does not carry the whole application: vendored framework code in one case, built frontend assets and `vendor/` in the other. That is tier 3 by the table above. The alternative is a package that does not run.

Open

- **A failure’s headline outranked its diagnosis for weeks.** Bugsink’s hand-written Nix recipe ran a two-process application as one process (fixed 2026-08-01, §What the numbers mean). Every occurrence printed both “failed to start within 240.0s” and, immediately below it, the correct diagnosis: “the server bound its socket but no worker is serving”. Three timeout increases were spent on the first line. Nobody read the second. The platform’s diagnostic was right and unread, which is a harder problem than a missing diagnostic; an argument for putting the specific finding *above* the generic timeout headline.
- **Docker is unclaimed.** Recipes exist and none has been measured at the sign-in bar; the reports no longer carry the variant. Both families that *have* been measured failed comprehensively on first contact, so this is a known unknown.
- **Easy!Appointments cannot be verified by either path.** Its bootstrap runs and reports success; the served page carries no form inputs for `check.py` to post, and the browser harness fills the JavaScript-built form and remains on it. It may need a check that queries the application directly.
- **Two applications are not photographed signed in:** Mattermost’s React login is not drivable by the harness, and Easy!Appointments fails outright. Paheko and Bugsink were on this list and have come off it; both now photograph, in every variant that installs. Invoice Ninja (Flutter canvas) and Radicale (HTTP Basic, rendering identically signed in or out) are declared undrivable, not counted as gaps. **Uptime Kuma joins them undeclared:** the 2026-08-01 run captured its sign-in page and no signed-in shot, and nobody has said whether that is a harness gap or another application the harness cannot drive. 50 of 55 variants are photographed.
- **Isso ships no built frontend** under Nix: the python-venv template has no frontend build phase, so its admin dashboard serves a 200 whose JavaScript 404s; a `contains` assertion passes on it.
- **[healthcheck].contains is declared by no recipe in the corpus,** and a deploy treats any status line as “serving”. The “App is now running” a deploy prints is therefore satisfied by a 500, which makes a deploy’s own report of success a weaker claim than these reports are.

Experience Report: BookStack

Simple, self-hosted documentation platform.

What this app exercised

Laravel under Nix, where PHP's `__DIR__` resolves symlinks and lands back in the read-only store; the first app to need the tree copied rather than linked, which is now `needs-writable-dir`. Also the corpus's clearest case of an application that authenticates on the email while offering a username.

What broke

The credential in the recipe was not the credential in the app. BookStack seeds `admin@admin.com / password` when it migrates, and for a long time nothing replaced it: the deployed instance had an administrator whose password is documented upstream, while the operator was handed a generated one that did not work.

A hidden migration failure. The hand-written Nix wrapper ran `artisan migrate --force 2>/dev/null || true`, so an app with no schema reported a successful start.

BookStack authenticates on the EMAIL, and username is only a display name. A credential reader handed both picks the username and is rejected as if the password were wrong; which is why the recipe declares `login = "email"`. The generated variant dropped that line and failed in exactly that way while its own `check.py`, which names the email field directly, kept passing.

What the platform gained

`needs-writable-dir`, and the `login` key in `[admin]`; which tells anything reading a credential which field the app actually authenticates on. `make-nix-variants.py` now refuses to generate a variant whose `sign-in` field differs from its native counterpart, because this app is where that drift was found.

Deployment variants

Native downloads the release and runs `composer install`; **Docker** builds from `debian:bookworm-slim`; **Nix** copies the tree writable (PHP resolves `__DIR__` through symlinks into the store) and generates `.env` from the addon's variables. The template variant uses `php-app` with `needs-writable-dir`.

Verification

`apps/bookstack/check.py` signs in with the `[admin]` credential, reaches a page only a session can, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator; the weaker claim, since that password can be changed out from under it.

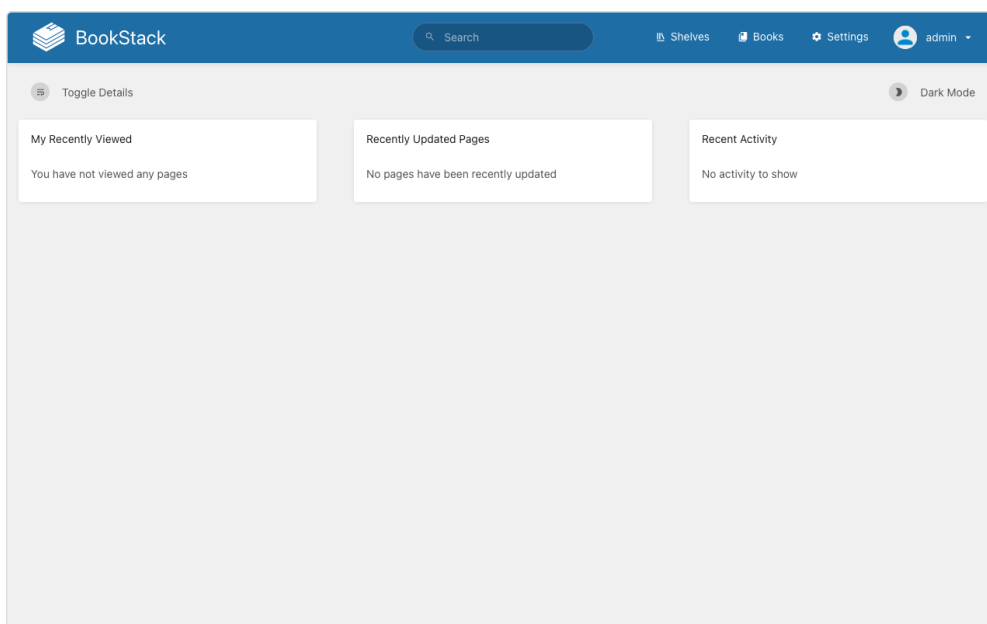
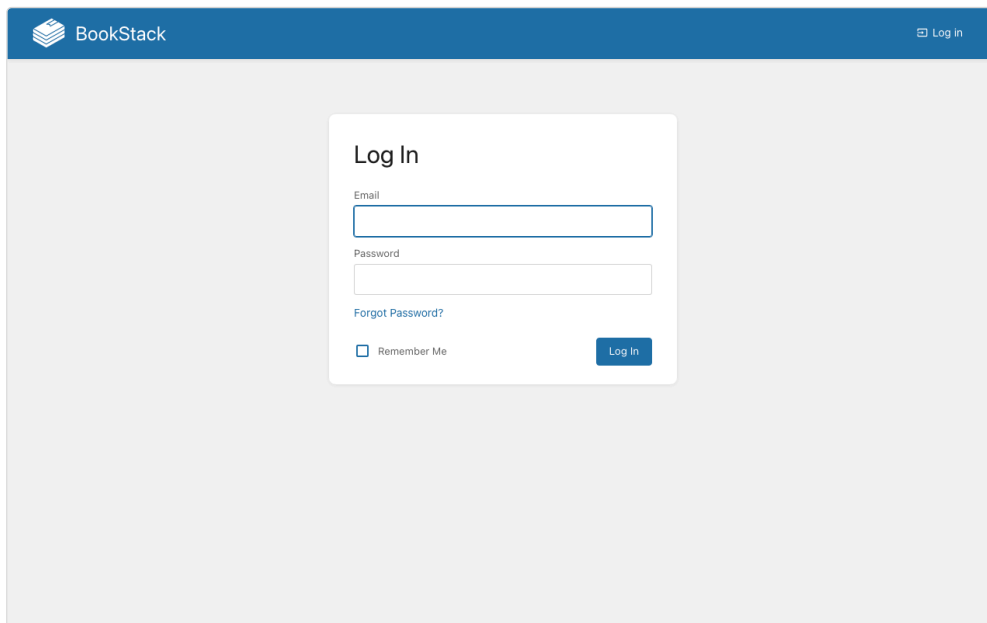
Reproduce

```
hop3 catalog install bookstack
hop3 app check --app bookstack
```

Open

Nothing open.

Screenshots



Experience Report: Bugsink

Self-hosted error tracking, Sentry SDK-compatible.

What this app exercised

Two processes from one recipe: a web worker and snappea, a background queue that keeps its own separate database. It is also the app that showed `[run.workers]` and a second `migrate --database=` are both needed before a post-login page works.

What broke

Everything below is from the two Nix variants; the native one has been clean.

The bootstrap had nowhere to run. A Nix app's build artifact reported the read-only store path as its working directory, so `[run] before-run` (where every native recipe keeps its first-run setup) could not be used at all. The recipe compensated with a `pre-exec` block in the generated wrapper that re-implemented part of the setup and skipped the rest.

Two different secrets, both unstable. `SECRET_KEY` was `$(head -c 32 /dev/urandom | base64)` in the wrapper's exports, which the wrapper re-evaluates on *every start*: a new Django signing key per restart, invalidating every session. Separately, `pre-exec` minted its own key into `bugsink_conf.py`, which lives under `src/` and is wiped and regenerated on each deploy.

No administrator was ever created, so the smoke test reported “the sign-in did not establish a session” against an application that was working correctly and simply had no account.

psycopg could not load libpq. Once `before-run` worked, `migrate` died with `ImproperlyConfigured: Error loading psycopg2 or psycopg module`. The recipe declares `nix-runtime-libs`, but that became a single `LD_LIBRARY_PATH` export *inside the wrapper*, so the app process had the libraries and nothing else did. The same failure then reappeared one layer along, in `[probe].create`, because the `create` commands did not receive the artifact's runtime environment either.

DATABASE_URL was composed in `[nix.env-exports]`, which is also wrapper-only. `[probe].create` had `PGUSER` and `PGHOST` but not the URL, and Django failed building a connection.

The hand-written Nix recipe ran the application as one process (found and fixed 2026-08-01, after the template variant was already clean). It carried neither half of the second process the other two recipes have (no `migrate --database=snappea` for the queue's own database, no `[run.workers] snappea`) while generating its config from the same `--template docker` that assumes both. It could not have carried them: its setup lived inside the *web* start wrapper, and whatever runs in that file runs only for `unicorn`, so setup was unshareable with a second worker by construction.

It presented as “*failed to start within 240.0s*”, so the response each time was to raise the timeout: 120 native, 180 nix-gen, 240 nix, three increases spent on it. It was not slow. The `unicorn` master bound the port eight seconds in, logged `Listening at`, and never logged `Booting worker`: no traceback, nothing served, a silent hang for the remaining 232 seconds. The platform said exactly that in the line beneath the headline (“*the app's port is listening but it did not answer an HTTP request: the server bound its socket but no worker is serving*”) at every occurrence. After the fix: 40 seconds to a pass.

What the platform gained

`before-run` for Nix applications now runs in the app's own directory with the artifact's runtime environment applied (`working_dir`, and `nix-runtime-libs` reaching more than the wrapper). Before that a Nix app had nowhere to run a first-run bootstrap, and `psycpg` could not load `libpq` outside the wrapper.

Cost

Most of the time went to two platform gaps rather than the app: `before-run` had no usable working directory for a Nix app, and the declared runtime libraries reached only the wrapper.

Deployment variants

Two processes in every variant: `gunicorn`, plus the `snappea` worker that drains a queue living in a second database. Nix vendors pure-Python `psycpg v3` (the binary wheels bake a hash-pinned `libkrb5` absent from a Nix-built `venv`) and emits `LD_LIBRARY_PATH` for the libraries it dlopens. It is the first consumer of `nix-runtime-libs`.

Verification

`apps/bugsink/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

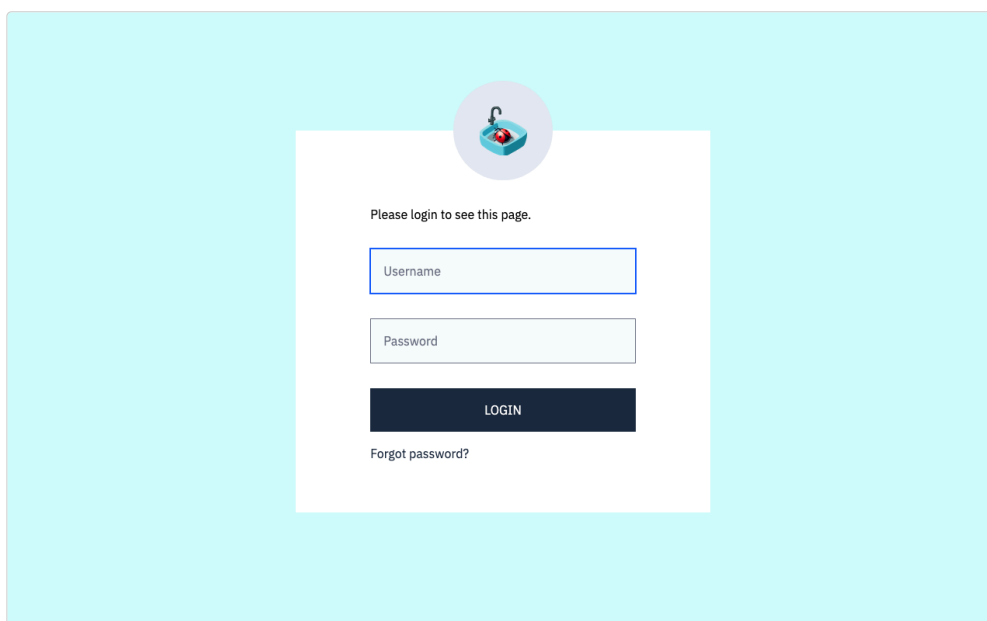
Reproduce

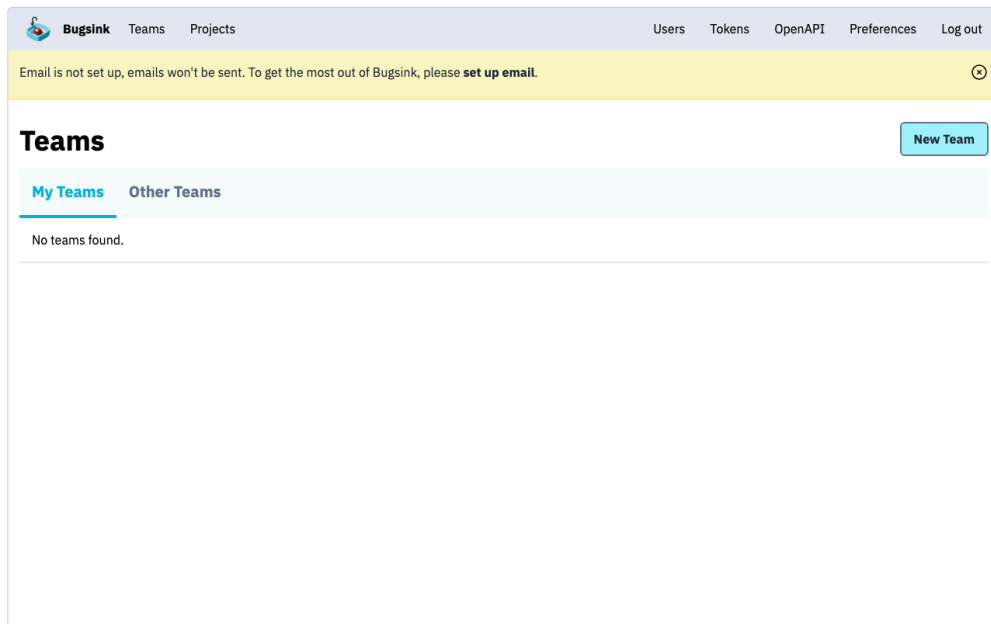
```
hop3 catalog install bugsink
hop3 app check --app bugsink
```

Open

No open items remain. All three variants sign in, and all three photograph. The hand-written Nix variant gained a signed-in shot once it stopped hanging, having previously had a login page only.

Screenshots





Experience Report: Dolibarr

Open source ERP and CRM for small and medium businesses.

What this app exercised

An application with **no installer CLI at all**. It was listed as a permanent deferral for that reason, and the question it forced (can a browser wizard be driven headlessly, or is a whole class of business software out of reach?) is the one it was kept for.

What broke

It was never installed. The application deployed, started, and answered every request (the login page included) with *“Dolibarr config file content seems to be not correctly defined. Please run dolibarr setup by calling page /install”*. Under Nix it was served straight out of the read-only store, so it could not have been installed even in principle.

Dolibarr ships **no installer CLI**, only a browser wizard, which is why it was once listed as a permanent deferral. Each wizard step also reads its inputs from \$argv under the PHP CLI, so the same steps a browser drives can be driven from a script; that is how it is installed now, in every variant.

Its steps can print an error and still exit 0, so the bootstrap checks for the admin row directly.

What the platform gained

Nothing in the platform; the answer was a technique. Driving a browser wizard’s steps through the PHP CLI is now the pattern for any application that ships no installer, and Matomo uses it too. Both were permanent deferrals before it.

Deployment variants

Its web root is htdocs, where most PHP frameworks use public. **Native** and both **Nix** variants drive the browser wizard’s steps from the CLI through the same script; **Docker** does the same at entrypoint. It uses Postgres, which makes it the corpus’s test of that addon against a PHP toolchain.

Verification

apps/dolibarr/check.py signs in with the [admin] credential, reaches a page only a session can, and confirms a wrong password is refused. It has no [probe] account, so it signs in as the operator’s administrator: the weaker claim, since that password can be changed out from under it.

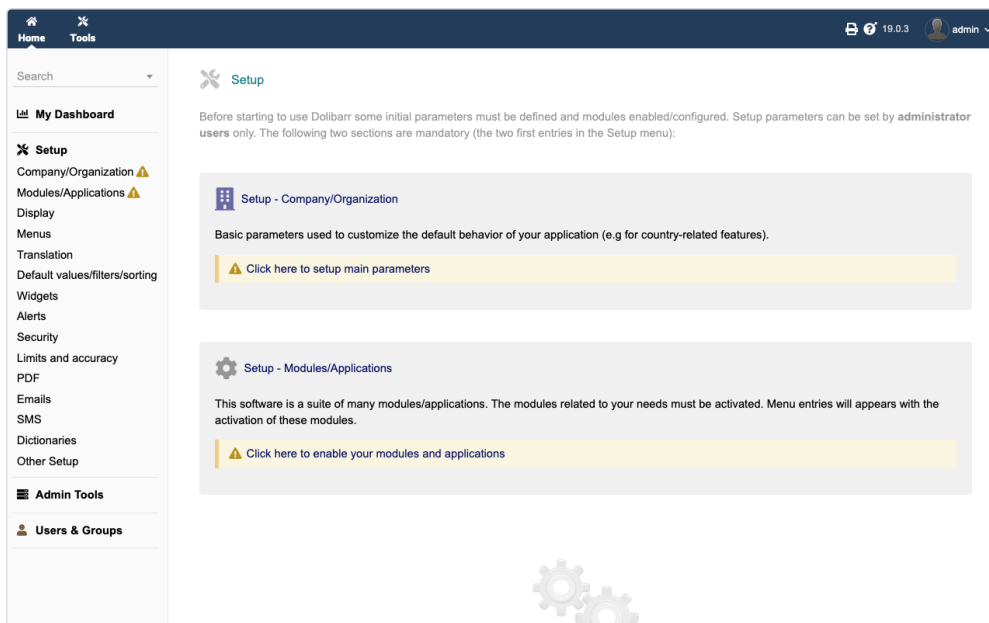
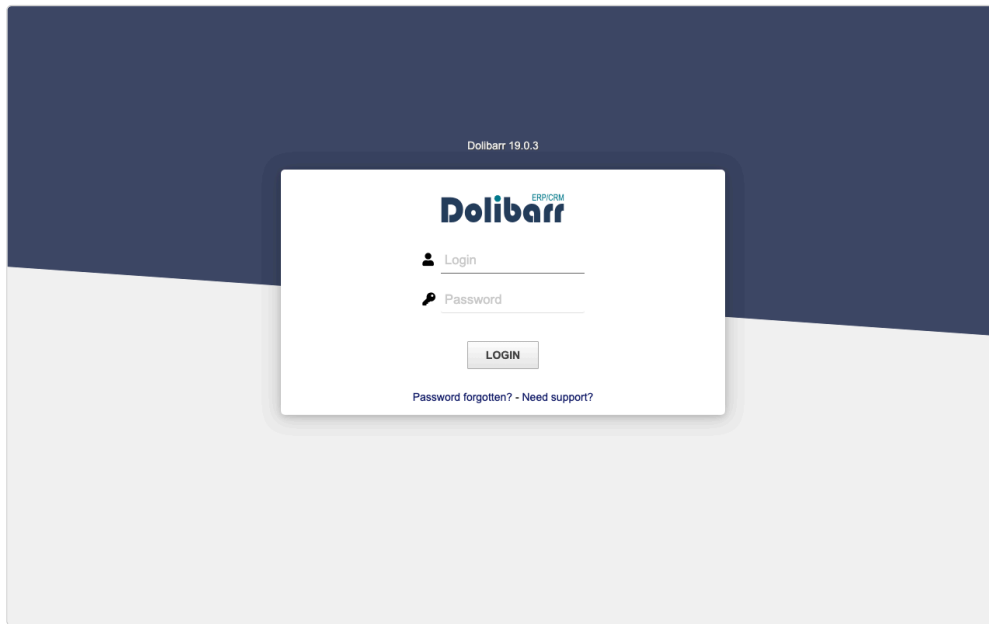
Reproduce

```
hop3 catalog install dolibarr
hop3 app check --app dolibarr
```

Open

Nothing open.

Screenshots



Experience Report: Easy!Appointments

Open source appointment scheduling.

What this app exercised

Whether the sign-in bar can be met by an application that builds its login form in JavaScript. On the template variant the answer so far is no, by either verification path.

What broke

The login form is built in JavaScript, so the served page carries no inputs at all. On the template variant an HTTP check cannot sign in, and it is the one variant in the corpus the browser harness cannot sign into either: it fills the form, submits, and is still looking at it afterwards. The bootstrap reports success and the credential is reconciled; nothing has demonstrated the result.

The git tag is not the release. It omits both the minified frontend assets and vendor/, so a package built from it is missing part of the application.

BASE_URL was built by hand as `http://${HOST_NAME}`: right host, wrong scheme. The app constructs its login POST from that constant, so on an HTTPS site the browser was asked to submit over a scheme the page cannot use.

What the platform gained

unzip on buildComposerProject's nativeBuildInputs when the source is a zip, in the php-app template. Otherwise this app is a standing question: the one variant in the corpus the sign-in bar cannot yet reach.

Deployment variants

Every variant takes the upstream *release* archive rather than the git tag. It is a flat zip, so source-root is ".", and it already ships 27,549 vendor/ entries, so composer is switched off.

Verification

apps/easy-appointments/check.py *attempts* to sign in with the [admin] credential and to confirm a wrong password is refused; on the template variant neither it nor the browser harness completes (see Open). It has no [probe] account, so what it attempts is the operator's administrator sign-in.

Reproduce

```
hop3 catalog install easy-appointments
hop3 app check --app easy-appointments
```

Open

- **nix-gen (fail)**: the bootstrap runs and reports success (the schema is created and the admin is reconciled to the injected credential) but neither verification path completes. check.py defers to the browser because the served page carries no form inputs at all, and the browser fills the JavaScript-

built form, submits, and is still looking at it afterwards. The application may well be correct; nothing has demonstrated it.

Screenshots

BACKEND SECTION

Welcome! You will need to login in order to view backend pages.

Username

Password

[Forgot Your Password?](#)

➔ LOGIN

Powered by [EasyAppointments](#)

EASY!APPOINTMENTS
ONLINE APPOINTMENT SCHEDULER

CALENDAR CUSTOMERS SERVICES USERS JOHN DOE

All

< > Today

JUL 26 – AUG 1, 2026

Day Week Month

	Sun 7/26	Mon 7/27	Tue 7/28	Wed 7/29	Thu 7/30	Fri 7/31	Sat 8/1
All Day							
7 am							
8 am							
9 am							
10 am							
11 am							
12 pm							

Experience Report: Forgejo

Self-hosted Git service (community fork of Gitea).

What this app exercised

Same shape as Gitea. Showed that a Go binary is named after its module path (`forgejo.org`), independent of the project name.

What broke

Four defects are shared with Gitea, whose report describes them: an admin bootstrap calling `./forgejo` in a layout where the binary lives in the store, `$out/bin` holding only the generated wrapper, three signing secrets minted by the wrapper and rotating on every restart, and open registration shipping in every Nix variant; the same fixes apply, and the same lesson about what the sign-in bar cannot catch.

The binary's name alone is Forgejo's own. `buildGoModule` names the output after the module path element, so the source build produces `forgejo.org` (as Miniflux's produces `miniflux.app`), and every `forgejo ...` invocation in the recipe had to be `forgejo.org ...`.

What the platform gained

Nothing beyond what Gitea contributed; the two share their defects and their fixes.

Deployment variants

Nix (hand-crafted) takes nixpkgs' `forgejo`, whose `generic.nix` renames the server binary to `gitea` in `preInstall`; **Nix (template-generated)** compiles from source with `go-source`, and `buildGoModule` names it after the module path element, `forgejo.org`. One application, two Nix packagings, two different binaries.

Verification

`apps/forgejo/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

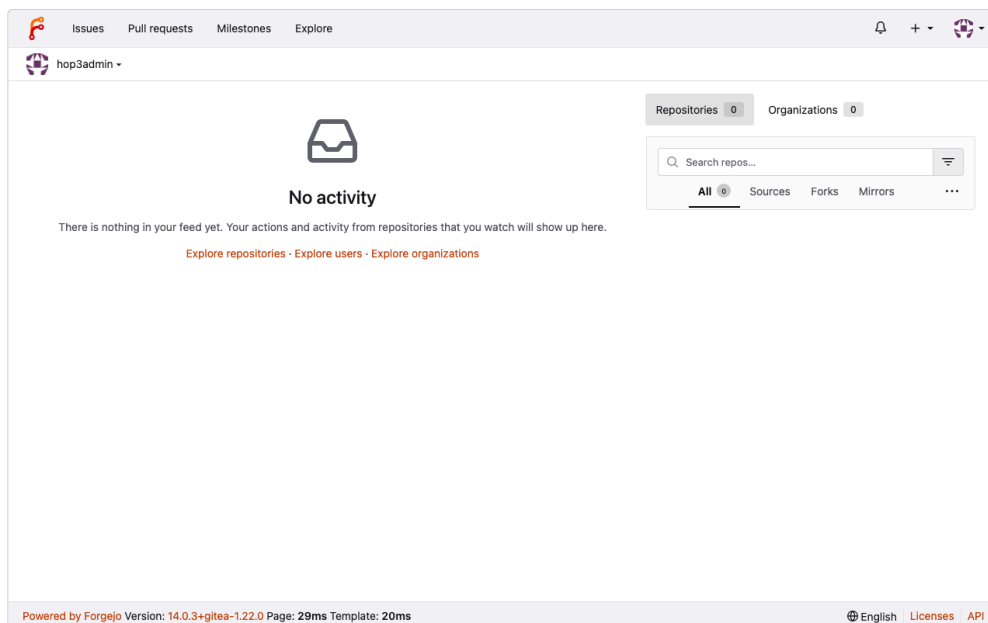
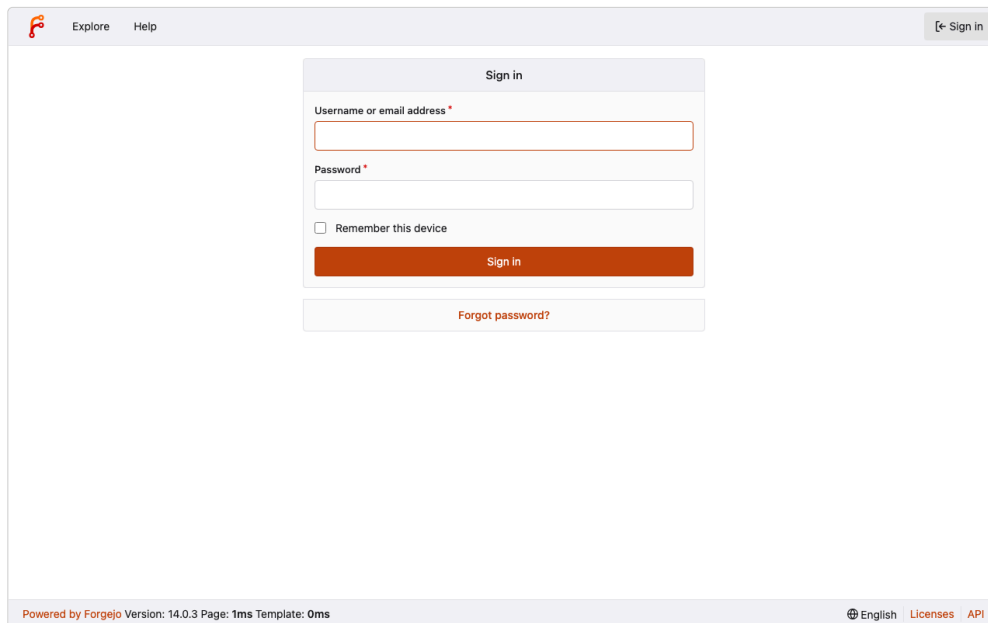
Reproduce

```
hop3 catalog install forgejo
hop3 app check --app forgejo
```

Open

Nothing open.

Screenshots



Experience Report: Gitea

Self-hosted Git service.

What this app exercised

A Go source build with a JavaScript frontend, and an app whose own CLI must be reachable to create the administrator. It is the first consumer of `go-static-dirs` for source assets that are not the compiled frontend.

What broke

The admin bootstrap named a path that does not exist. `[admin].create` came from the native recipe as `./gitea admin user create`, which is correct in a source tree holding the binary and meaningless under Nix. Both Nix variants failed with `sh: ./gitea: not found` after a full build.

Three signing secrets rotated on every restart. `SECRET_KEY`, `INTERNAL_TOKEN` and `JWT_SECRET` were minted with `$(head -c 32 /dev/urandom | base64)` inside a config file the wrapper rewrites at each start. `SECRET_KEY` encrypts 2FA secrets and stored credentials, `INTERNAL_TOKEN` authenticates git hooks to the web process, `JWT_SECRET` signs OAuth2 grants. Rotating them makes encrypted data undecryptable and invalidates every token, silently, on a restart nobody connected to it.

ROOT_URL was `http://localhost`, which is what Gitea puts in clone URLs, redirects and OAuth callbacks.

Two traps are Gitea's own, shared with Forgejo: `admin` is a reserved username it refuses, and `admin user create` prints the refusal while **exiting 0**. Verifying the account exists is the only reliable check.

Open registration shipped in every Nix variant. `DISABLE_REGISTRATION = true` lives in the native recipes' shell scripts, and no Nix variant carries a `scripts/` directory; so both Nix builds put an internet-facing forge online on which the first visitor could register. It is now declared in the config each variant generates, and a `GET /user/sign_up` on a deployed instance answers with the disabled notice.

The sign-in bar does not catch this: an application with open registration signs in perfectly, refuses a wrong password, and passes every check in the corpus. Reading the native recipe beside the Nix one found it, the same method that closed the last four failures; it is the clearest evidence that the bar is a floor.

What the platform gained

`${pkg}`, a stable binding for the application's own derivation, so a recipe can put the app's CLI on `PATH` without knowing the app id. `$out/bin` holds only the generated wrapper.

Deployment variants

Native downloads the release binary; **Nix (hand-crafted)** wraps `nixpkgs' gitea`; **Nix (template-generated)** compiles from source with `go-source`, building the JavaScript frontend alongside. All three generate `custom/conf/app.ini` from the addon's variables.

Verification

`apps/gitea/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

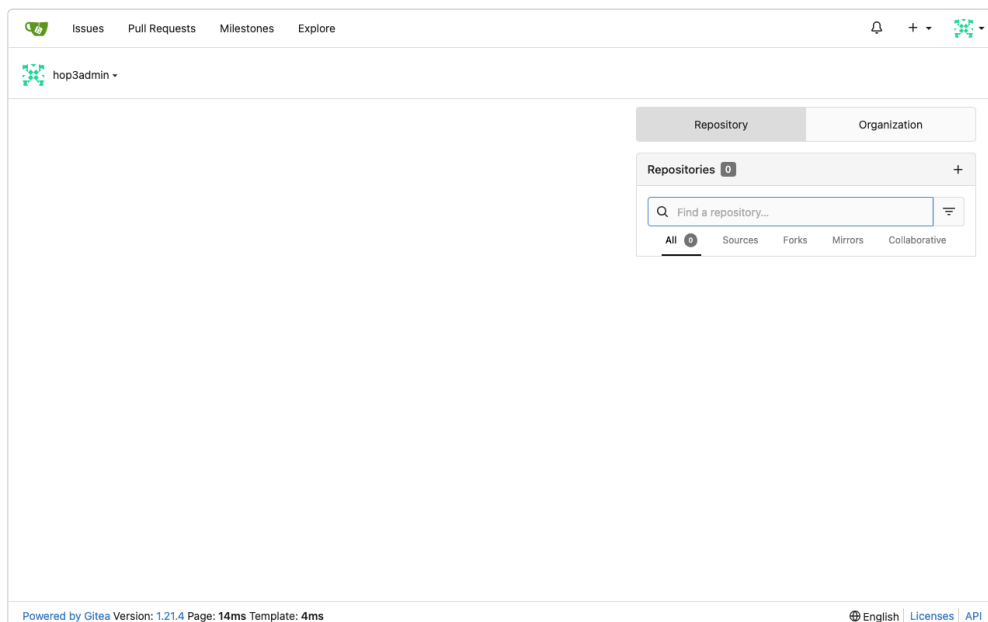
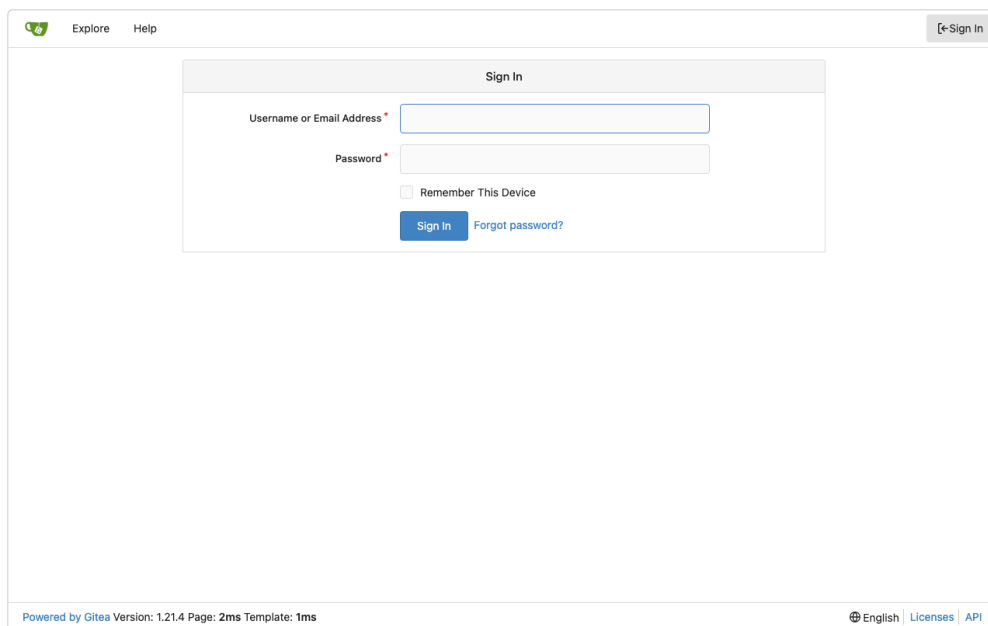
Reproduce

```
hop3 catalog install gitea
hop3 app check --app gitea
```

Open

Nothing open.

Screenshots



Experience Report: Invoice Ninja

Free open-source invoicing platform.

What this app exercised

A Laravel application whose UI is a Flutter canvas, so nothing a selector can reach. It probes the boundary of browser-driven verification. The boundary is declared; workarounds stop there.

What broke

No account was ever created, so `POST /api/v1/login` answered 401 on an application that looked deployed and healthy. The migration that should have preceded it ran as `artisan migrate --force 2>/dev/null || true`, hiding its own failure.

APP_URL pinned to localhost put / in a redirect loop. Laravel derives the scheme and host of every redirect it issues from that value; Chromium gave up with `ERR_TOO_MANY_REDIRECTS` while the recipe's own contains assertion on `/login` passed throughout.

Invoice Ninja authenticates on the email, and the generated variant had acquired a username the native recipe deliberately omits, which a credential reader then prefers.

What the platform gained

The screenshot harness's unsupported declaration: a way to say *this application cannot be driven by a browser, and here is why*. It isn't a gap. Radicale uses it for a different reason.

Deployment variants

Native rebuilds the frontend with npm; the **Nix** variants rely on the committed Flutter bundle. Each variant ships something different, and both pass.

Verification

`apps/invoice-ninja/check.py` signs in with the `[admin]` credential, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator: the weaker claim, since that password can be changed out from under it.

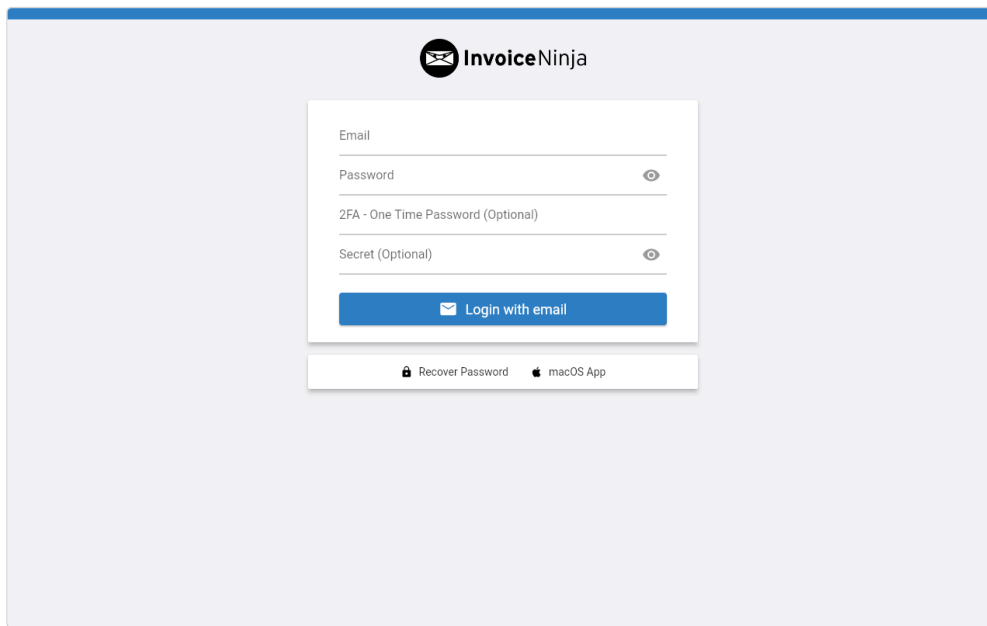
Reproduce

```
hop3 catalog install invoice-ninja
hop3 app check --app invoice-ninja
```

Open

- **nix, nix-gen:** the sign-in page is photographed; there is no signed-in shot. The Flutter canvas offers no DOM inputs a selector can reach: declared unsupported, not a gap. `check.py` signs in over HTTP and is unaffected.

Screenshots



Experience Report: Isso

A lightweight commenting system, Disqus alternative.

What this app exercised

A comment widget whose / is an API. Its moderation dashboard is password-only: there is no account to name.

What broke

It was served with its moderation dashboard disabled. The hand-written Nix config had no [admin] section at all, so /admin/ answered without ever asking for a password. The check reported “*the admin dashboard is not asking for a password*”, and it was right: there was no dashboard to ask. Isso has no admin username, so the password is the whole credential; its absence leaves the dashboard open.

Comments were self-publishing. Without [moderation] enabled, any anonymous visitor’s comment appears immediately, which is isso’s equivalent of open registration.

Its JavaScript 404s. The python-venv template has no frontend build phase, so the admin dashboard’s assets are not built. The page answers 200 and a contains assertion passes on it. Content assertions are not verification.

What the platform gained

No platform change; the contribution is evidential, and it is the one above: the argument for the sign-in bar, made in one line of config.

Deployment variants

Not in python3Packages, so every variant pip-installs it; the Nix ones do so into a sealed venv from a hash-pinned requirement set. SQLite and no addon, which makes it the lowest-friction app in the corpus and a useful control.

Verification

apps/isso/check.py signs in with the [admin] credential, and confirms a wrong password is refused. It has no [probe] account, so it signs in as the operator’s administrator: the weaker claim, since that password can be changed out from under it.

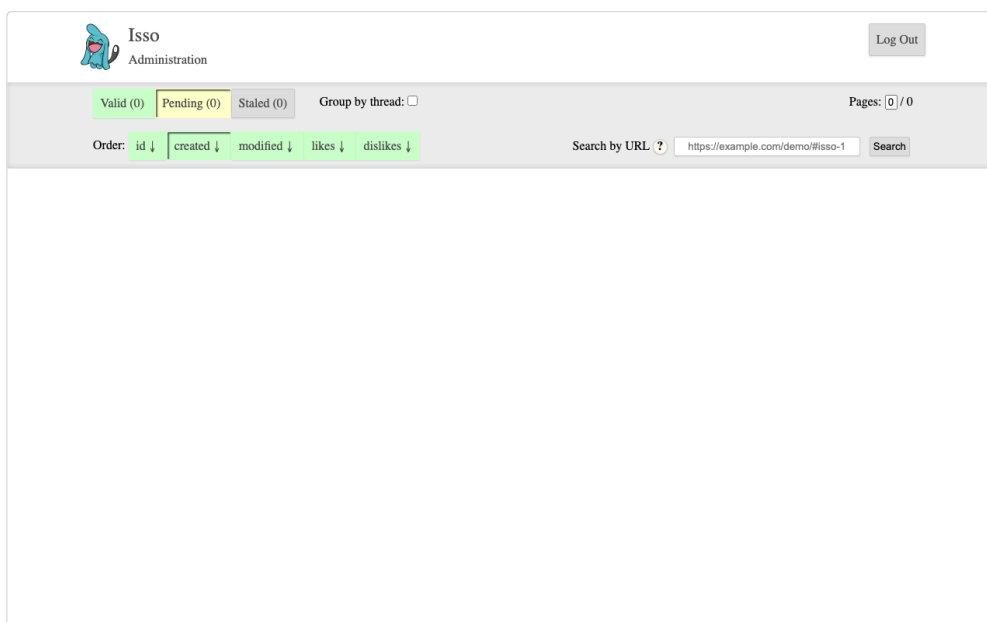
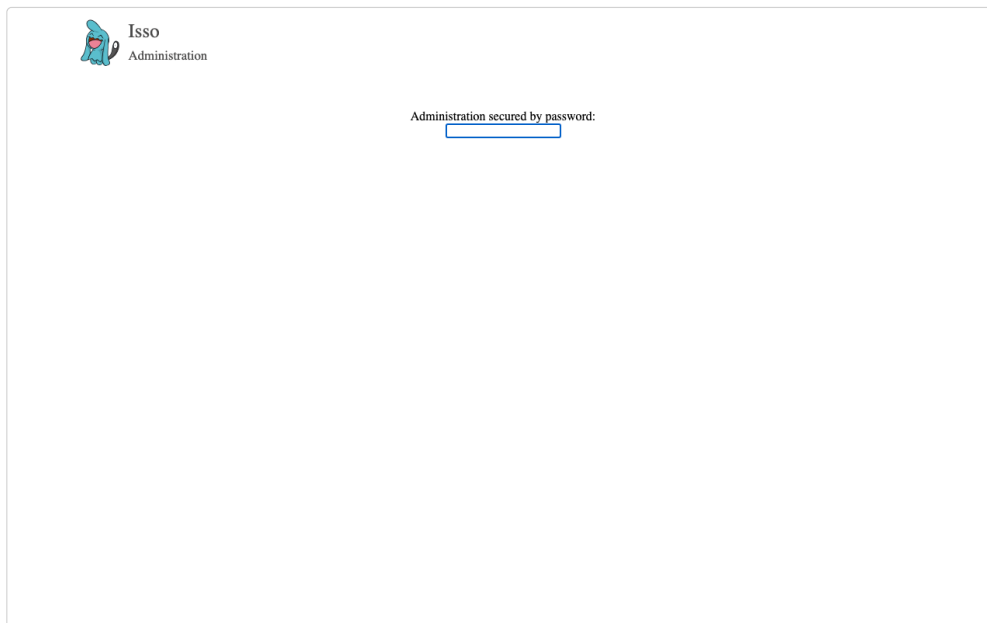
Reproduce

```
hop3 catalog install isso
hop3 app check --app isso
```

Open

Nothing open.

Screenshots



Experience Report: Kanboard

Kanban project management software.

What this app exercised

The plainest PHP application in the set, with no composer and no build step, which makes it a control: anything that breaks here points to the platform. It earned that role by breaking twice, both times in the transport.

What broke

It was served out of the read-only store, so it could not write its database, its config or its uploads, and had no account anyone could sign in as.

A PHP warning killed the session. Kanboard's default log driver writes to a file and warns *to the response* when it cannot: output ahead of the headers, which suppresses Set-Cookie. The login POST then has no session to carry, and Kanboard reports that as **"The username is required"**: an application-level message describing a transport-level failure. Pinning `LOG_DRIVER = stderr` is the fix, and the working variants had it all along.

There is no `cli user:reset-password` subcommand. The call to one sat inside a `|| true` and did nothing at all, leaving the shipped `admin/admin` in place. The password has to be written through Kanboard's own container.

PLUGIN_INSTALLER defaults on, which lets a signed-in administrator install arbitrary remote code.

What the platform gained

No platform change; the control role above is the contribution. Its lasting artefact is the example that keeps being useful: "The username is required" describing a suppressed Set-Cookie.

Deployment variants

No composer, no build step: the simplest PHP app here. Every variant writes `config.php` from the addon's variables and serves with PHP's built-in server. The Nix ones copy the tree writable first, since Kanboard writes its database, uploads and sessions inside it.

Verification

`apps/kanboard/check.py` signs in with the `[admin]` credential, reaches a page only a session can, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator: the weaker claim, since that password can be changed out from under it.

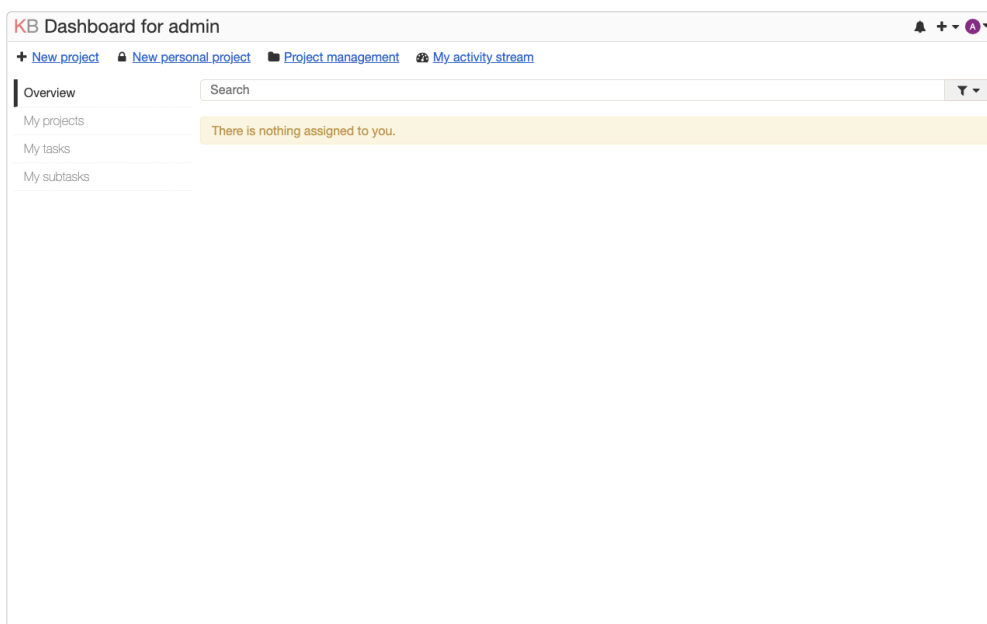
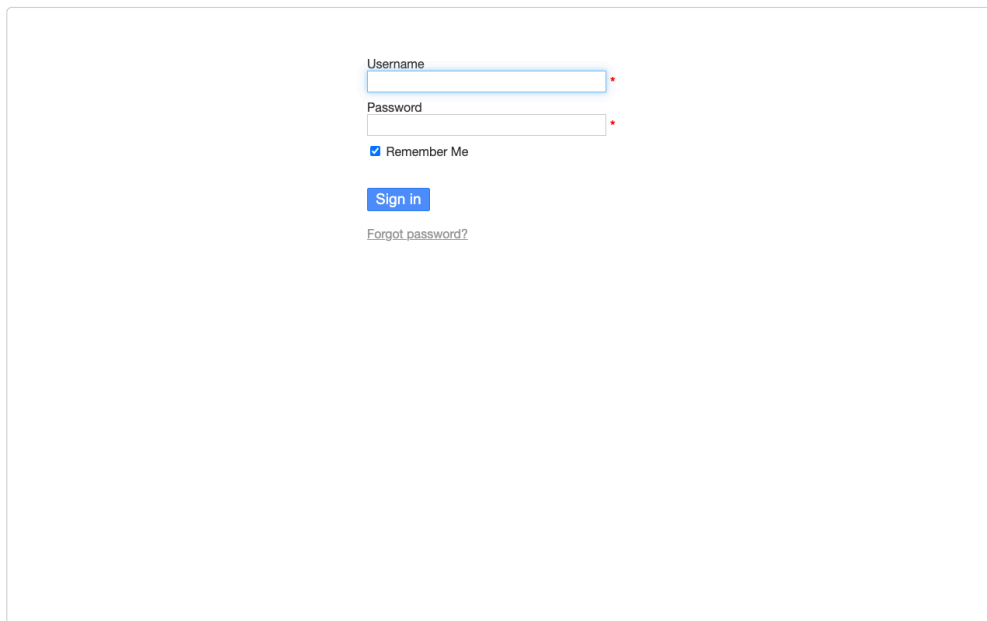
Reproduce

```
hop3 catalog install kanboard
hop3 app check --app kanboard
```

Open

Nothing open.

Screenshots



Experience Report: Keycloak

Enterprise SSO / OIDC / SAML identity and access management.

What this app exercised

An identity provider with no login form to post: its check obtains an OIDC token instead. Also the first consumer of `writable-home-at-runtime`.

What broke

The generated Nix expression did not evaluate. `extra-paths` referenced `${keycloak}`, but the let-binding is derived from the app id, so in the `keycloak-nixgen` variant it is `keycloak_nixgen`. The recipe had been made by copying and renaming, which renamed the binding underneath it. Nix failed at *build* time with undefined variable `'keycloak'`, pointing at a generated line nobody wrote.

The admin console's sign-in redirect pointed nowhere reachable. `KC_HOSTNAME` was absent, so Keycloak built its OIDC issuer, redirect URIs and console asset paths from the address it believes it is served on (behind nginx, `http://0.0.0.0:<port>`). The console loaded and never rendered a form, while `/realms/master`, which needs none of that, answered 200 throughout. It needs the public URL and `KC_PROXY_HEADERS=xforwarded` so the scheme Keycloak uses matches the one nginx terminated.

The realm shipped with a published administrator password. `KC_BOOTSTRAP_ADMIN_PASSWORD = "changeme"` was a literal in the recipe, and nothing mapped Hop3's generated credential onto it. The deployed instance had an administrator whose password is in the repository, while the operator was handed one that did not work, and the smoke test reported only the second half of that.

What the platform gained

`writable-home-at-runtime` and `[nix.env-exports-raw]`, both built for it and both since used elsewhere. This app established the escape hatches' purpose.

Deployment variants

The hardest consumer of the Nix escape hatches. Quarkus rewrites `lib/quarkus` on first boot, so the store copy is lazily copied into a writable home; `nixpkgs' kc.sh` is a compiled Go wrapper hardcoding a path back into the read-only store, so the raw upstream script is exec'd instead; and that script falls back to `java` on `PATH` unless `JAVA_HOME` is interpolated at Nix build time, which is what `[nix.env-exports-raw]` exists for.

Verification

`apps/keycloak/check.py` authenticates as the `[probe]` account, which Hop3 owns and rotates (by obtaining an OIDC token, since Keycloak serves no login form to post), and confirms a wrong password fails to obtain one.

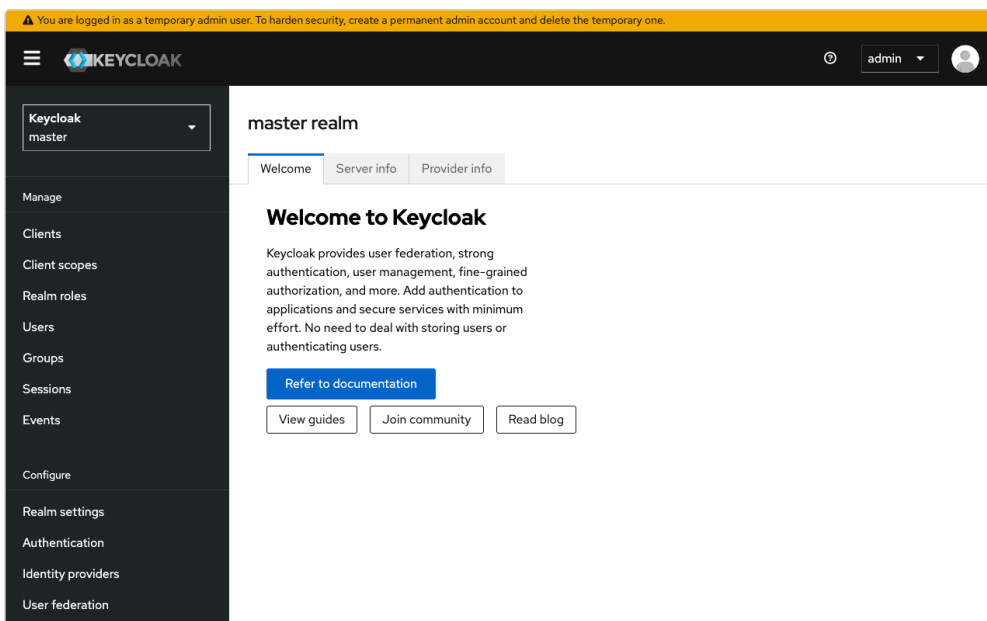
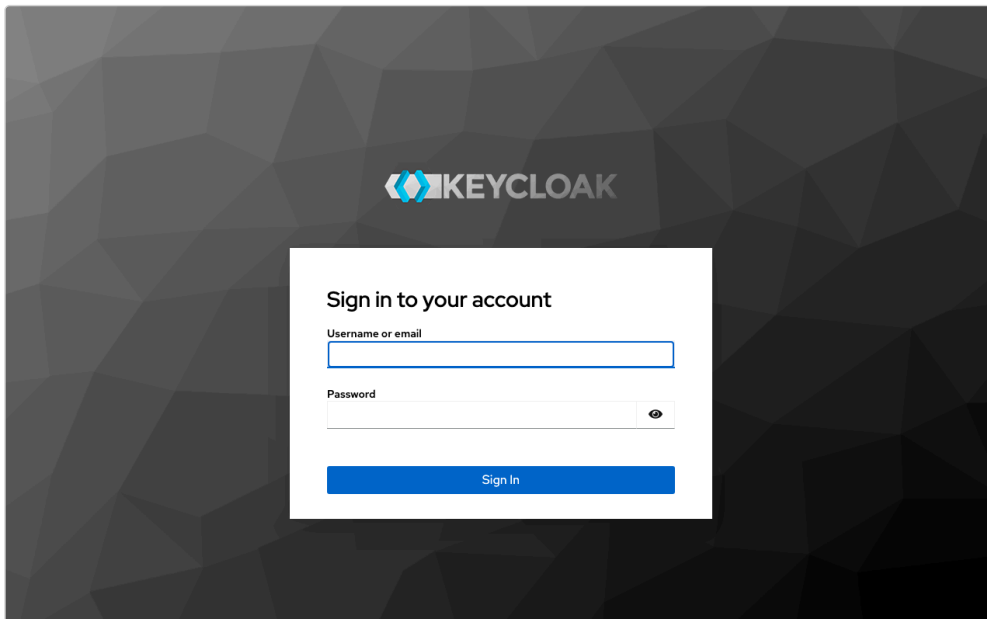
Reproduce

```
hop3 catalog install keycloak
hop3 app check --app keycloak
```

Open

Nothing open.

Screenshots



Experience Report: LimeSurvey

Professional online survey and data collection tool.

What this app exercised

A JavaScript-rendered admin UI that cannot be signed into over HTTP, so verification has to be deferred to a browser. It is the reason a run without `--screenshots` reports some apps as unverifiable, outside the pass/fail count.

What broke

The recipe installed it with a published password. `console.php install admin password123 Admin admin@example.com`, with the result discarded by `2>/dev/null || true`: a known credential on every deployment *and* a failed install reported as success, in one line.

Its admin UI is rendered by JavaScript, so a form POST returns LimeSurvey's "JavaScript deactivated" notice and no HTTP sign-in is possible; verification defers to the browser, as above.

The URL format determines whether the login form appears. Without `urlManager` set to `urlFormat: path` and `showScriptName: true`, LimeSurvey builds its URLs in a form where the JavaScript that renders the login is fetched from paths that do not resolve: the page loads and the form never appears.

What the platform gained

The runner's vocabulary for *unverifiable*: an application whose sign-in only a browser can drive is reported as such. A login page being served is not enough to count it as a pass.

Deployment variants

Unlike most of the PHP set, this uses Postgres. Every variant installs through `application/commands/console.php install`, which creates the schema and the administrator in one call; there is no separate migration step and the credential mapping is the whole bootstrap.

Verification

`apps/limesurvey/check.py` signs in with the `[admin]` credential, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator: the weaker claim, since that password can be changed out from under it.

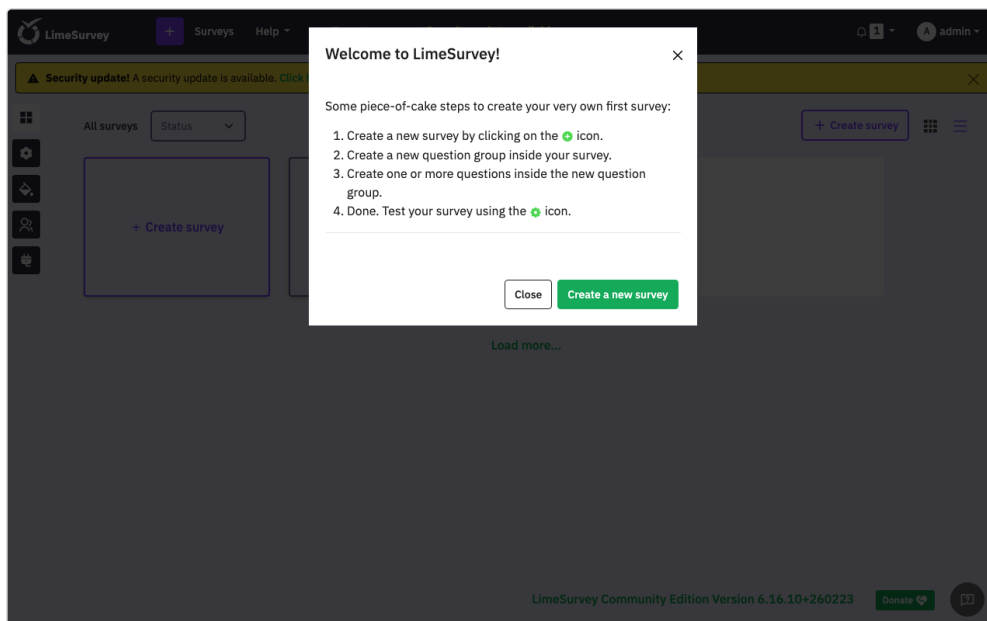
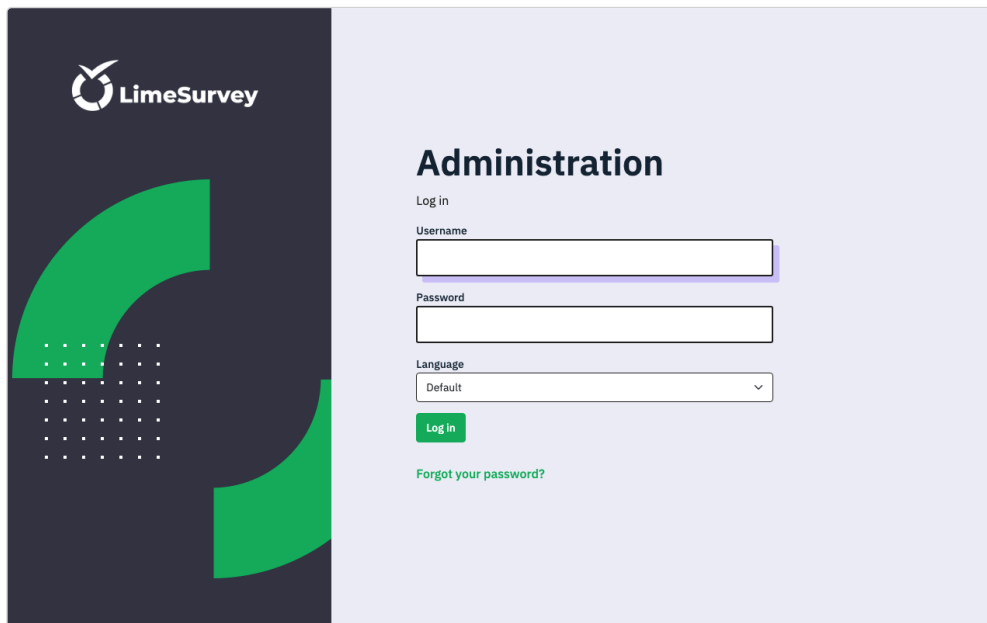
Reproduce

```
hop3 catalog install limesurvey
hop3 app check --app limesurvey
```

Open

Nothing open.

Screenshots



Experience Report: Matomo

Open source web analytics platform.

What this app exercised

An application with no user-creation CLI. Its installer is a browser wizard, so the account is made by driving Matomo's own model classes from PHP

What broke

Matomo 5 has no installation CLI. Its Installation plugin is a browser wizard, and there is no console command for it or for creating a superuser. The app was once listed as a permanent deferral for that reason. The headless installer written for it performs the same steps the wizard performs, each guarded separately so it is idempotent *and* resumable: a run that created the schema and then failed skips the schema on the next deploy and goes on. Checking only “do the tables exist?” would declare a half-installed database finished.

Installing is not enough. The schema the installer writes is not necessarily at the code's version, and until `console core:update` has run Matomo answers every request with a 302 to `?module=CoreUpdater`, including the login page a check posts to.

Without a stable salt, every redeploy signs every visitor out; it is what Matomo signs sessions and cookies with.

A marker that could never match. Its check keyed on `module=Login&action=logout`, and the served HTML escapes the ampersand. The assertion looked exact and was unsatisfiable.

What the platform gained

`[probe].create` is now required. A probe account Hop3 cannot create is one it can never offer to a check; the optional form silently did nothing.

Deployment variants

Every variant runs the same headless installer, because Matomo ships none. **Native** and **Docker** call it from `before-run`; the **Nix** ones from the wrapper, after the tree is writable. All follow it with `console core:update`.

Verification

`apps/matomo/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

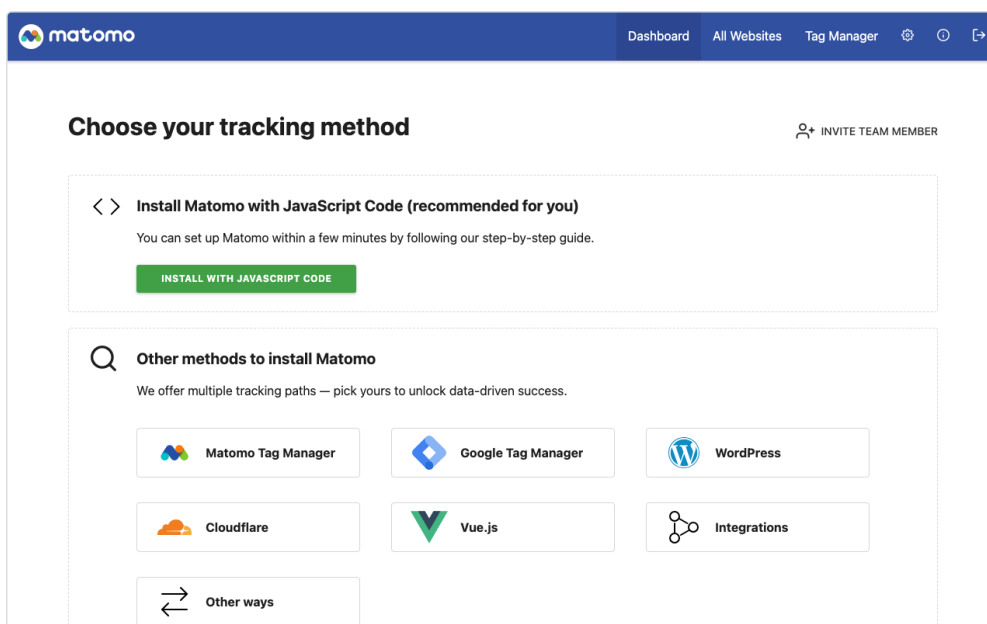
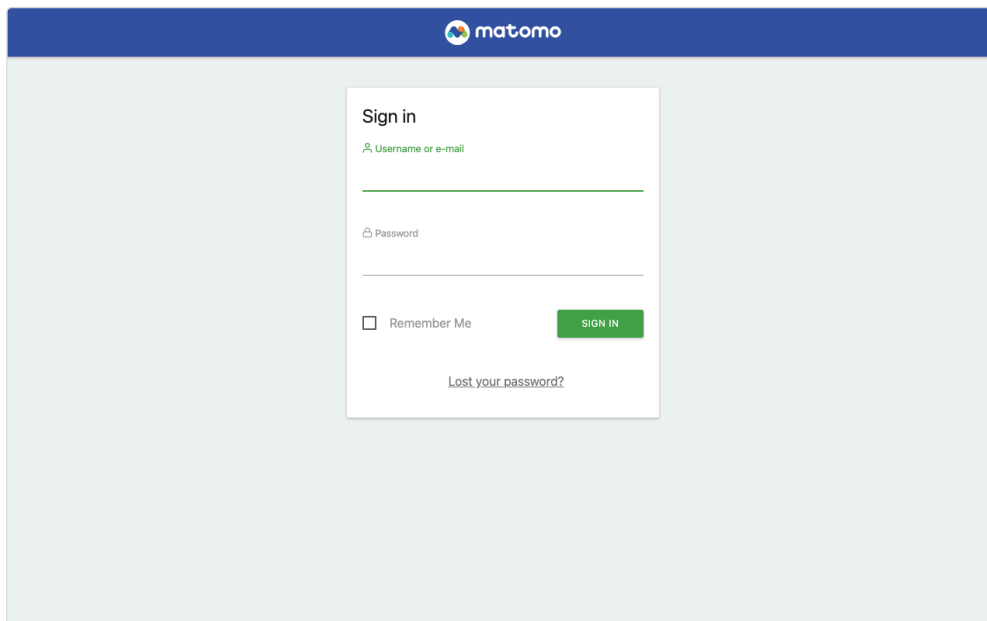
Reproduce

```
hop3 catalog install matomo
hop3 app check --app matomo
```

Open

Nothing open.

Screenshots



Experience Report: Mattermost

Open source team collaboration platform.

What this app exercised

A nixpkgs derivation does not carry the tools its application's documentation assumes.

What broke

Every `mmctl --local` call in the account bootstrap exited 127. The mattermost package's `bin/` holds only `mattermost`; `mmctl` is a separate package and has to be asked for by name.

`SiteURL` pinned to `localhost` left the webapp loading against an origin the visitor does not have. The REST check (a POST to `/api/v4/users/login`) passed throughout, so the failure was visible only to a browser.

Its assets live beside the binary, which under Nix means the read-only store; they have to be linked into the writable working directory before the server starts.

What the platform gained

`[nix.let-extra]`, which lets a recipe pull a second nixpkgs package into its derivation. Vikunja and Keycloak both use it now.

Deployment variants

Nix (hand-crafted) wraps nixpkgs' mattermost and pulls `mmctl` in as a second package; Nix (template-generated) does the same through `[nix.let-extra]`. Both link the shipped assets into the working directory, since Mattermost resolves them relative to its binary.

Verification

`apps/mattermost/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, and confirms a wrong password is refused.

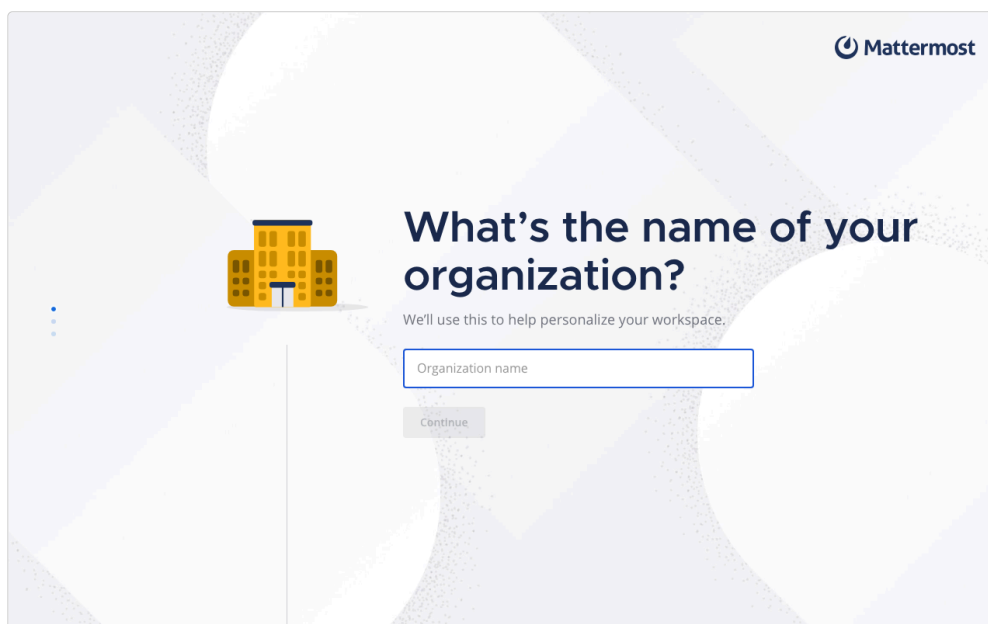
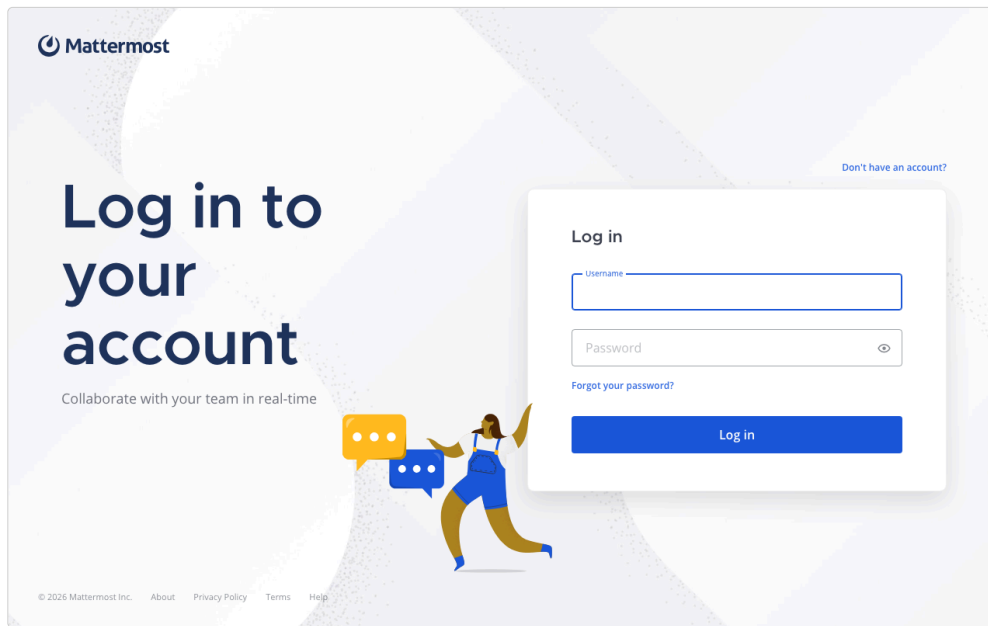
Reproduce

```
hop3 catalog install mattermost
hop3 app check --app mattermost
```

Open

- **nix, nix-gen:** no screenshot. The browser harness finds no password field at `/login` within 15 s; with `SiteURL` corrected, the “open in the app or the browser” interstitial it knew how to click through no longer appears, so that route's output remains undiagnosed. The sign-in itself is verified over the REST API.

Screenshots



Experience Report: Miniflux

Minimalist and opinionated RSS reader.

What this app exercised

An app that creates its own administrator at first start from injected environment variables, with no command for Hop3 to run.

What broke

It shipped a published administrator password. `ADMIN_PASSWORD="${ADMIN_PASSWORD:-changeme}"` appeared in three places in the hand-written Nix recipe (the wrapper, `runtime.json`, and the derivation's `env attrset`), so removing it from one changed nothing. Miniflux creates its administrator at first start from `CREATE_ADMIN` plus those variables, so there was no post-deploy command to fix it afterwards.

`buildGoModule` names the binary after the module path element, so the artefact is `miniflux.app`. Nothing in the app id predicts that.

What the platform gained

No platform change; a pattern: where an application creates its administrator at first start from environment variables, the credential mapping is the bootstrap, and it has to be right before the first start.

Deployment variants

A single binary configured entirely through environment variables: `DATABASE_URL` and little else. **Native** builds from source with `make`; the **Nix** variants take `nixpkgs`' package and compile from source respectively, which makes this the cleanest side-by-side of the two Nix strategies in the corpus.

Verification

`apps/miniflux/check.py` signs in with the `[admin]` credential, reaches a page only a session can, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator: the weaker claim, since that password can be changed out from under it.

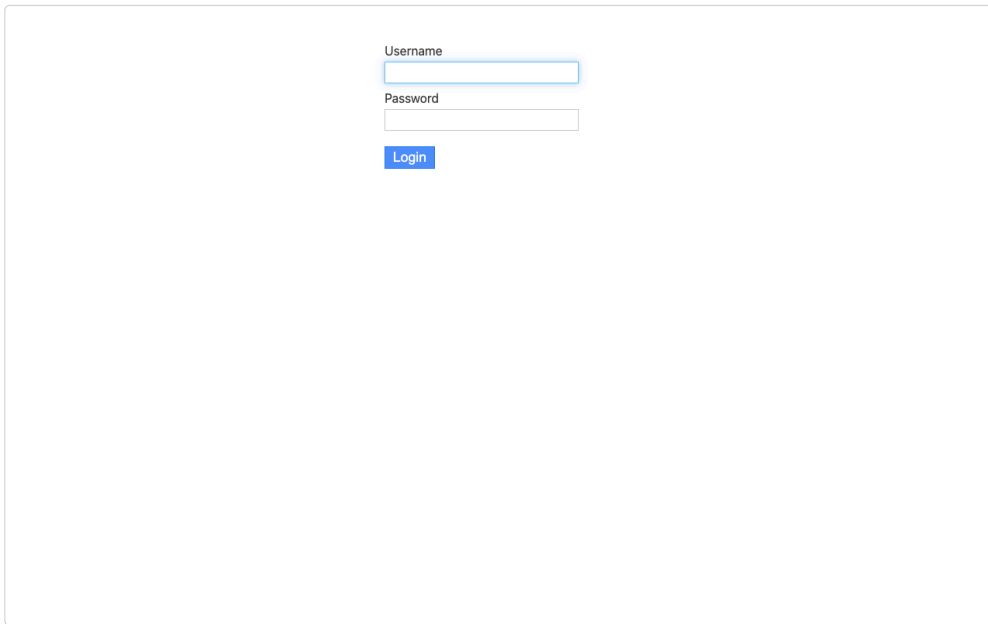
Reproduce

```
hop3 catalog install miniflux
hop3 app check --app miniflux
```

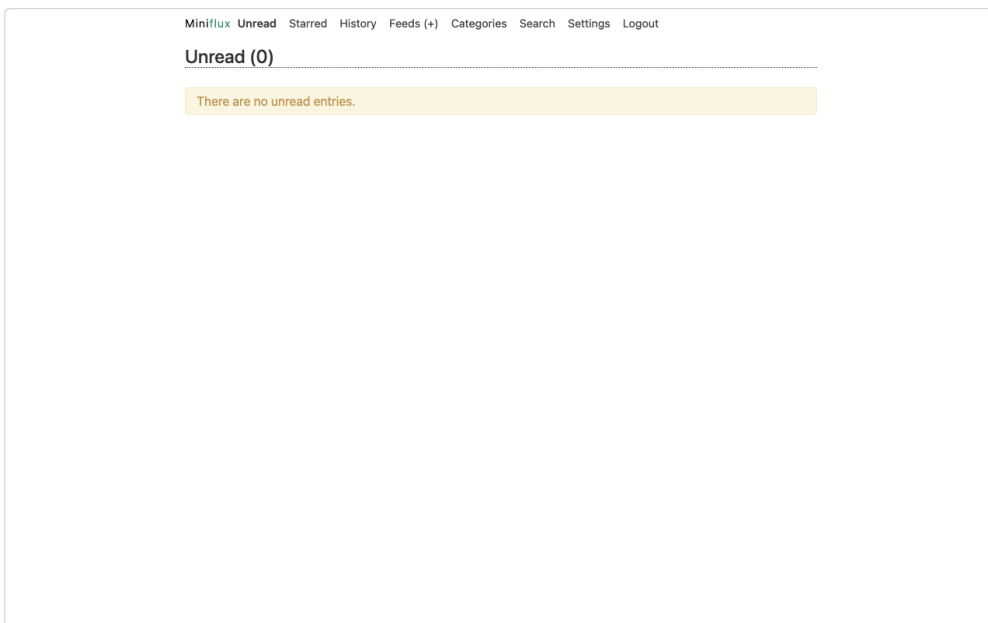
Open

Nothing open.

Screenshots



A screenshot of a login form. It features two input fields: the first is labeled "Username" and the second is labeled "Password". Below the password field is a blue button labeled "Login". The form is centered on a light gray background.



A screenshot of the "Unread" page in the Miniflux application. At the top, a navigation bar includes links for "Miniflux", "Unread", "Starred", "History", "Feeds (+)", "Categories", "Search", "Settings", and "Logout". Below the navigation bar, the heading "Unread (0)" is displayed. A yellow message box below the heading states "There are no unread entries."

Experience Report: Nextcloud

Self-hosted productivity platform.

What this app exercised

The heaviest PHP dependency surface in the corpus (extensions, a cron worker, background jobs) and the app whose failing bootstrap first showed that `admin.create` reporting an exit status is not enough to diagnose anything.

What broke

It answered GET /login with 400 because it was never installed: the writable tree was materialised when the app started, which is after `before-run`, so a correctly ported bootstrap ran in a directory that did not yet hold the application.

occ user:add takes no --email. The recipe passed one, the command failed, and the failure was invisible from the outside: Hop3 had already generated and displayed a credential for an account that did not exist. After this, `admin.create` began reporting its full command output.

It keeps a hidden password input in its signed-in markup, which had the screenshot harness call a successful sign-in a refusal. Counting only *visible* fields is what a person looking at the image would do.

What the platform gained

`admin.create` reports its command's output, which revealed the `occ user:add --email` mistake. The screenshot harness counts only *visible* password fields, because Nextcloud keeps a hidden one in its signed-in markup.

Deployment variants

The most extension-hungry app in the set, and under Nix each one is declared explicitly. `occ maintenance:install` drives first-run setup in every variant. **Docker** diverges deliberately: Postgres plus Redis where the others use MySQL, as a test of addon divergence across variants.

Verification

`apps/nextcloud/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

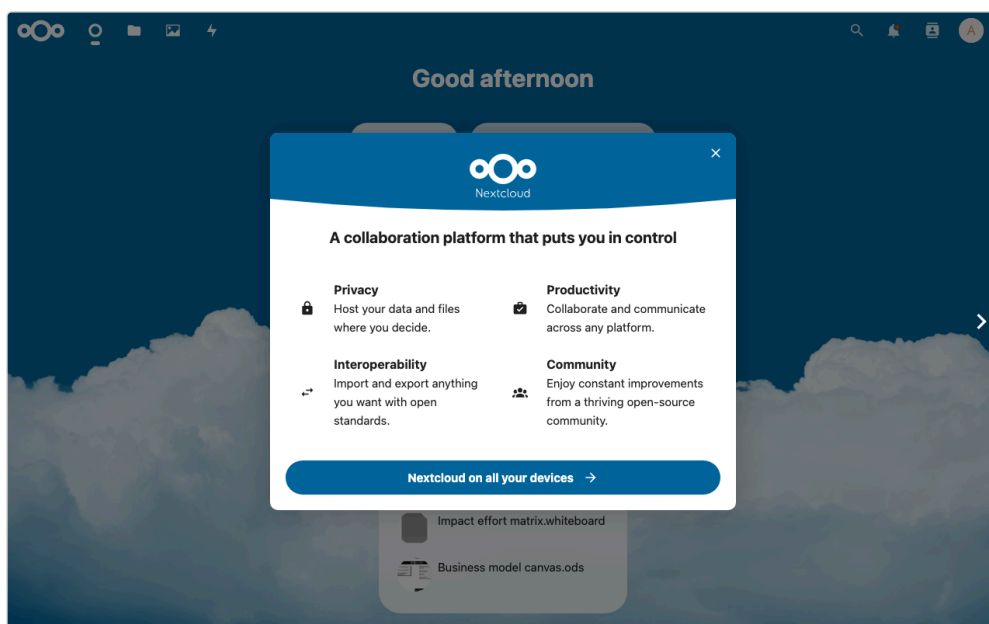
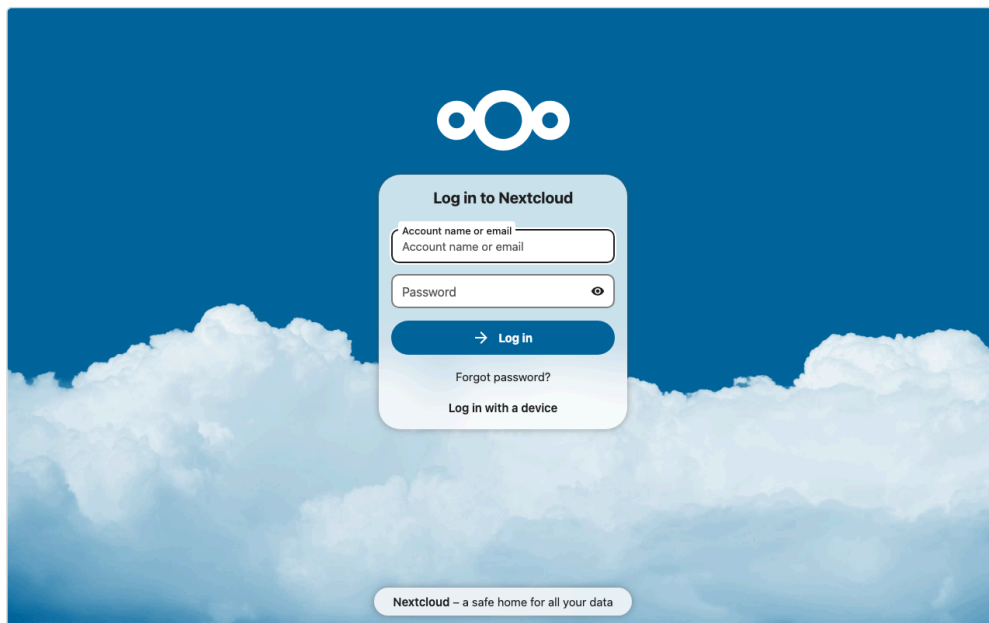
Reproduce

```
hop3 catalog install nextcloud
hop3 app check --app nextcloud
```

Open

Nothing open.

Screenshots



Experience Report: Paheko

Nonprofit accounting and association management.

What this app exercised

A PHP application that resolves its own public URL from `$_SERVER`, which is wrong behind a TLS-terminating proxy, and whose login form routes on the submit button's name.

What broke

The login form was never processed. The sign-in POST returned 200 with the form re-rendered and no error text. A correct password, a wrong password, and a deliberately *omitted* CSRF token all produced identical responses: a form that inspects its input returns different errors for different failures; this one returned none. The cause: Paheko routes on the submit control's name (`$form→runIf('login', ...)`), and the check posted only the form's hidden fields. A browser sends the button it clicked. The check omitted it, so `runIf` never matched.

It advertised itself over plain HTTP. Paheko resolves its own public URL from `$_SERVER`, and behind Hop3 it is reached over HTTP by the proxy that terminates TLS; it rendered `data-url="http://..."` on an HTTPS deployment. `HOP3_PUBLIC_URL` is injected for exactly this and the recipe was not using it. Fixing it did not fix the sign-in. It was a real defect and a wrong hypothesis at the same time.

The Nix package was missing part of the application. Both Nix variants died at every start on `require_once .../KD2/EventManager.php`: No such file or directory. Paheko vendors the KD2 framework into its release tarball and does not commit it, so a package built from the git tag is missing those files. Only the release archive supplies them. The release archive also has a flat layout where the tag put everything under `src/`.

Its config declared global constants. With the application finally present, `config.local.php` was writing bare `const DATA_ROOT` and friends, but Paheko reads `Paheko\DB_FILE`, so every setting was silently ignored and its defaults won. The file needs `namespace Paheko;` and guarded `defined() || define()`, because it is loaded with a plain `require` and a second bootstrap in one process turns the redefinition warning into a fatal.

What the platform gained

The checking library's form handling: `form_fields` now sends the submit control, because Paheko routes on which button was pressed and a browser sends the one it clicked. Every app's check benefited.

Deployment variants

Every variant takes the upstream *release* archive, for the reason above: only it carries the vendored KD2 framework. SQLite, no addon.

Verification

`apps/paheko/check.py` signs in with the `[admin]` credential, reaches a page only a session can, and confirms a wrong password is refused. It has no `[probe]` account, so it signs in as the operator's administrator: the weaker claim, since that password can be changed out from under it.

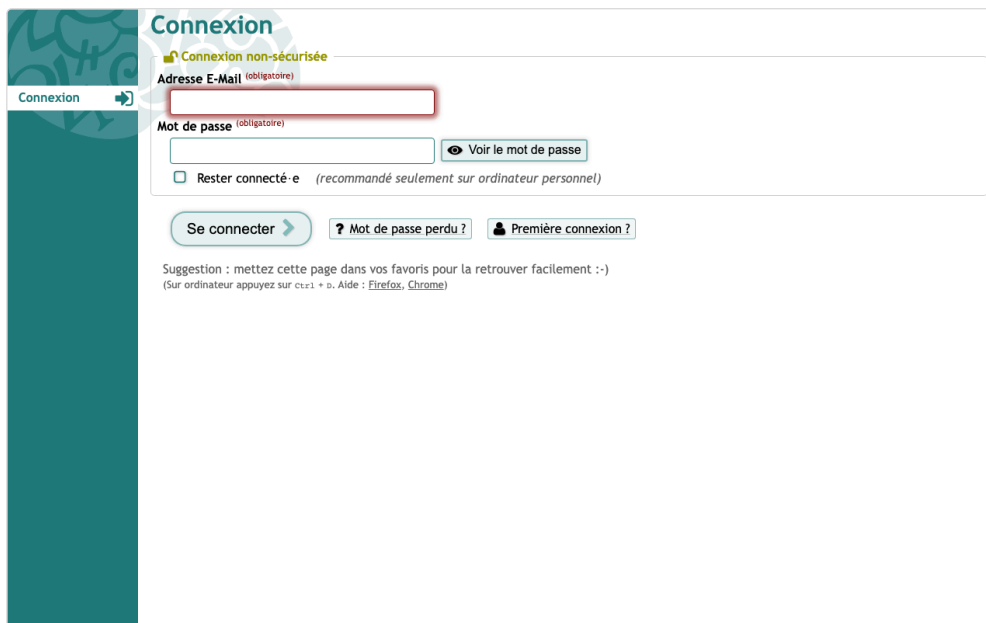
Reproduce

```
hop3 catalog install paheko
hop3 app check --app paheko
```

Open

- **nix**: no screenshot: the capture hangs on a ServiceWorker registration, as it does for the template-generated variant. The sign-in is verified over HTTP
- **nix-gen**: no screenshot. The capture times out after 60 s with no requests outstanding and a failed ServiceWorker registration in the page errors: the renderer is stuck, and resource starvation was ruled out. The sign-in itself is verified over HTTP

Screenshots



Experience Report: Radicale

A simple CalDAV and CardDAV server.

What this app exercised

Authentication with no user store: an account is a line in an htpasswd file, so both accounts are created by writing that file. Also the app that showed a file the server reads at startup cannot be created by a post-deploy command.

What broke

It served every calendar and address book to anyone who asked. [auth] type was `${RADICALE_AUTH_TYPE:-none}` and nothing ever set it otherwise, so the deployed server authenticated nobody. Every instrument the project had reported it green, because a server that authenticates nobody answers every request successfully. The only assertion that could catch it did: *a wrong password returned 200, not 401*.

The config was also written once and left alone, so a deployment that had ever run open stayed open for the rest of its life. It is rewritten on every start now, and the auth type is spelled out where no environment can switch it off.

There is no signed-in screenshot to take. Radicale authenticates over HTTP Basic and its `.web` interface renders identically whether or not a credential was sent; the two images were once byte-identical and were being counted as two.

What the platform gained

The screenshot harness no longer photographs a page it has not signed into.

Deployment variants

Available as a top-level nixpkgs package. The Nix variants wrap it without a build step. File-based storage and no addon. Accounts are lines in an htpasswd file (there is no user CLI), so `create` writes a bcrypt hash directly, which is why the derivation puts a `python3` carrying `bcrypt` on the app's `PATH` in its own right.

Verification

`apps/radicale/check.py` signs in with the [probe] account, which Hop3 owns and rotates, and confirms a wrong password is refused.

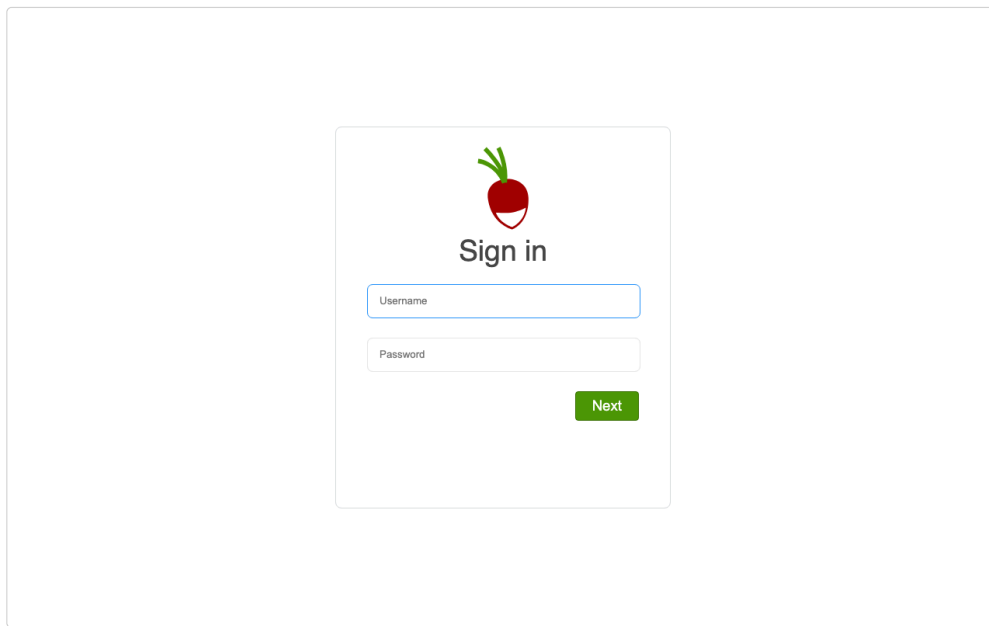
Reproduce

```
hop3 catalog install radicale
hop3 app check --app radicale
```

Open

- **nix, nix-gen:** the sign-in page is photographed; there is no signed-in shot, deliberately (see above).

Screenshots



Only the sign-in page, deliberately: a signed-in image would be identical (see above).

Experience Report: Uptime Kuma

Self-hosted uptime monitoring with a web UI and status pages.

What this app exercised

An app that authenticates over socket.io, so its check drives the application's own bundled client through node.

What broke

The smoke test accused a working application for a week. It reported “the credential Hop3 generated was refused”; the credential signs in through a browser without trouble. The probe drives the app's own bundled socket.io client through node, and every wait in it resolved on its own timeout:

- the connect wait resolved whether or not a socket opened;
- the next wait left offered as `null`, and the check asked only whether it was `'setup': null` is not `'setup'`, so “the server offers a login, not its setup wizard” passed **without a socket ever being opened**;
- the login then timed out, and a timeout was returned as `false`, which the check rendered as a refusal.

Three silent fallbacks stacked into an accusation. Making them fail loudly also fixed the app: the probe had been emitting `login` before the server announced `loginRequired`, and a wait that must succeed cannot race ahead of it.

The probe account was created only when the admin was. `createProbeUser()` sat after an early return in the branch that creates the administrator, so any instance with an existing user row got no probe, undetectably, because nothing verified it.

The JavaScript lived inside a Python string. The probe was an embedded literal in `check.py`; a `\` in the Python source arrived at node as a bare quote and killed the whole probe with a `SyntaxError`, after a full deploy. It is a `scripts/probe.js` file now, which is what `node --check` can read.

What the platform gained

The catalog driver's separation of “the run could not verify this” from “this application failed” (the vocabulary LimeSurvey's browser-only sign-in introduced) hardened from the opposite direction: a probe whose silent fallbacks were converting “unverified” into a false “failed”.

Deployment variants

Node, no addon, and no template-generated variant. It authenticates over socket.io, so its check drives the app's own bundled client through node; the only application in the corpus verified that way.

Verification

`apps/uptime-kuma/check.py` signs in as the `[probe]` account, which Hop3 owns and rotates, over socket.io (driving the application's own bundled client through node, `scripts/probe.js`, since Uptime Kuma has no form to post) and confirms a wrong password is refused.

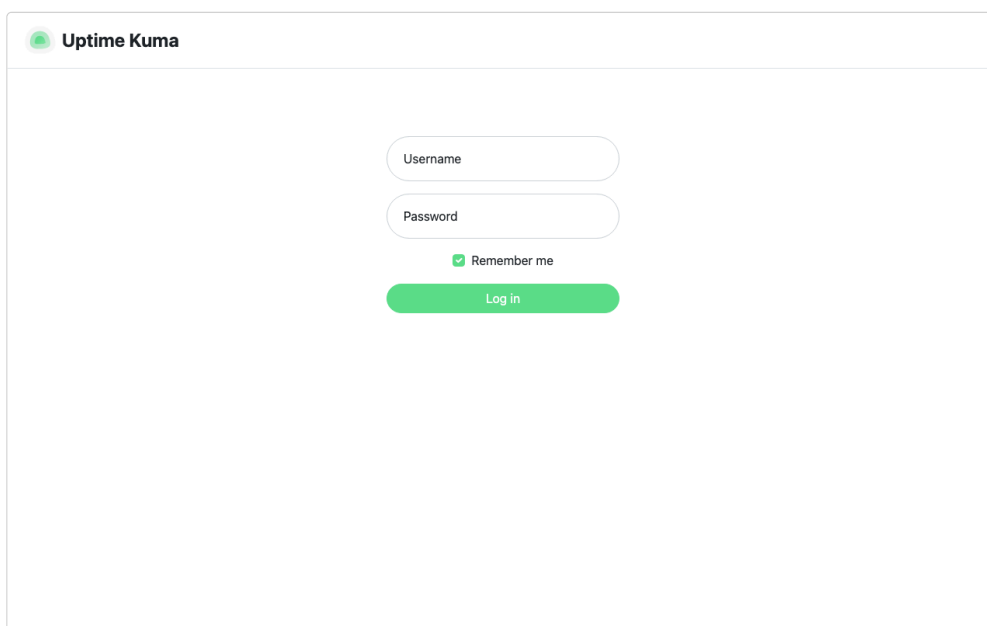
Reproduce

```
hop3 catalog install uptime-kuma  
hop3 app check --app uptime-kuma
```

Open

- **No signed-in screenshot.** Uptime Kuma authenticates over socket.io, so the browser harness cannot drive it. Its sign-in is verified by the node probe.
- **Only one variant exists.** Native is the whole of this app's coverage: there is no hand-written Nix recipe and no template one.

Screenshots



Experience Report: Vikunja

Open source task and project management.

What this app exercised

Two toolchains in one derivation: a Vue frontend built offline from a committed lockfile and embedded into a Go binary at compile time. `go-frontend-pnpm` is validated. Wrapper-only environment is invisible to the commands that bootstrap an app.

What broke

The accounts were created against the wrong database. Every setting was an `export VIKUNJA_*` inside the generated wrapper, so only the server process saw it; `[admin].create` runs outside the wrapper. `vikunja user create` therefore found no database configuration, said so in its own log on every run (“*No config file found, using default or config from environment variables*”), and wrote the accounts to Vikunja’s defaults instead of the app’s Postgres. A sign-in then answered “*Wrong username or password*” for the admin and 412 for the probe: two symptoms, one cause, and a day spent treating them as two problems.

`publicurl` is what the browser is told to call. Pinned to localhost, the Vue frontend aimed its API calls at the visitor’s own machine, failed to reach `/api/v1/info`, and rendered an error where the login form should be, while the application answered its own API check perfectly.

The binary is not named after the app. Built from source it is `api`; from `nixpkgs` it is `vikunja`.

What the platform gained

`go-frontend-pnpm`, and the `${pkg}` binding it shares with the forges. Anything exported inside the generated wrapper is invisible to `before-run` and to `create`.

Deployment variants

The only recipe here that builds two toolchains into one derivation: the Vue frontend is built offline from a committed `pnpm` lockfile and copied into `frontend/dist` before the Go compile, because Vikunja `go:embeds` it. **Nix (hand-crafted)** takes `nixpkgs`’ package instead: the same 0.24.6, differently built, with the binary named `vikunja` where the source build produces `api`.

Verification

`apps/vikunja/check.py` signs in with the `[probe]` account, which Hop3 owns and rotates, and confirms a wrong password is refused.

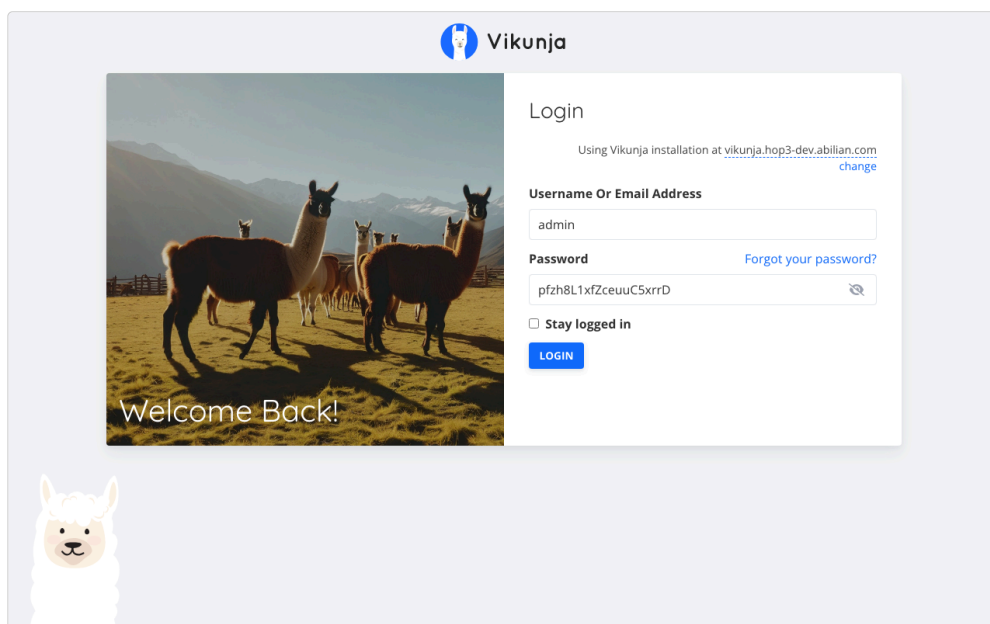
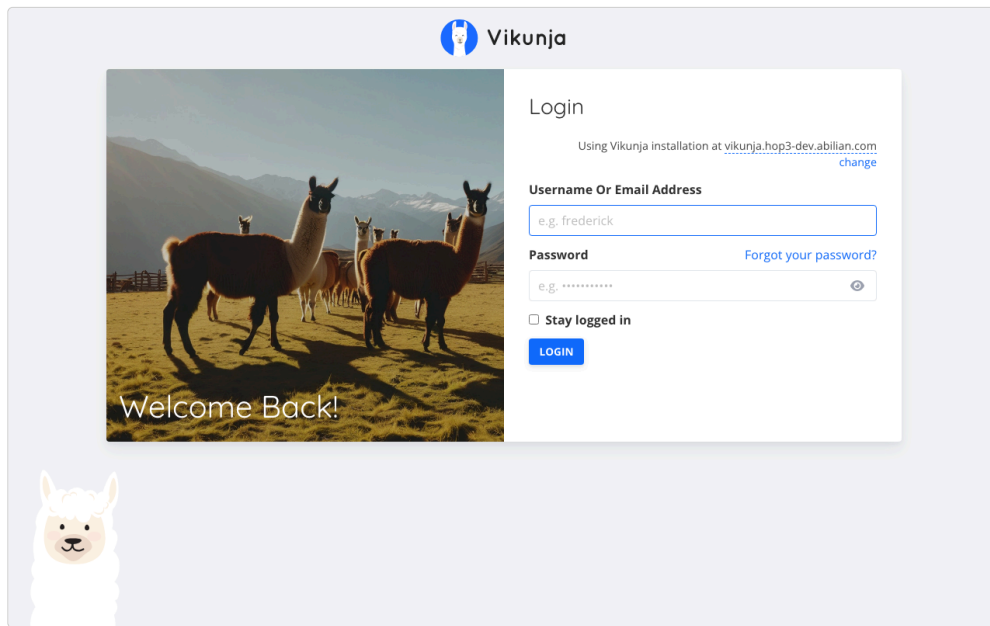
Reproduce

```
hop3 catalog install vikunja
hop3 app check --app vikunja
```

Open

Nothing open.

Screenshots



Experience Report: WordPress

Popular open source content management system.

What this app exercised

Nothing new. It is the control that says the common path is boring: `getenv()` configuration, no build step, and nothing any variant had to work around. A corpus needs one of these to make the others legible.

What broke

Nothing broke that was WordPress's doing. It reads its configuration through `getenv()` in `wp-config.php`, which makes it unusually clean to express in every build path, and the template variant needed nothing the others did not.

The one failure was ordering, and it was ours: the Docker variant started Apache before MySQL was accepting connections and returned a 500. A wait loop fixed it.

What the platform gained

The start-before-database race, first seen here, has since recurred across enough recipes to qualify as a platform pattern: a service-readiness wait belongs in the platform. Otherwise, a confirmation.

Deployment variants

No composer and no build step in any variant. `wp-config.php` reads through `getenv()`, so configuration is an environment bridge, and `wp core install` creates the administrator.

Verification

`apps/wordpress/check.py` signs in with the [probe] account, which Hop3 owns and rotates, reaches a page only a session can, and confirms a wrong password is refused.

Reproduce

```
hop3 catalog install wordpress
hop3 app check --app wordpress
```

Open

Nothing open.

Screenshots

